

ספר לימוד והדרכה מעשית

MAUI Learning Book

המדריך המעשי לפיתוח אפליקציות מובייל ודסקטופ ב-.NET MAUI

פרויקט מלווה: PlayListSample (.NET MAUI & SQLite)

ארכיטקטורה: MVVM, Service Layer & Repository Pattern

טכנולוגיות: C# 12, .NET 10, XAML, SQLite-net-pcl, xUnit

שנת מהדורה: 2026

MAUI Learning Book - PlayListSample

זהו ספר הלימוד המלווה את הפרויקט **PlayListSample**. הספר מסביר את העקרונות האדריכליים, החלטות התכנון, מתודולוגיית העבודה הנכונה לתלמיד, וכיצד כל שכבה פועלת בצורה נקייה וידידותית.

פרק 1: מתודולוגיית פיתוח – מאיפה מתחילים? View-First מול ViewModel-First

1.1 הדילמה של המפתח המתחיל

נניח שיש לנו רעיון לאפליקציה ויש לנו כבר את מודל הנתונים (`VideoItem`). ברגע זה כל תלמיד ומפתח שואל את עצמו: **לאן פונים עכשיו?**

- האם קודם כל פותחים את קובץ ה-XAML ומעצבים את המסך (View-First)?
 - או שקודם כל כותבים את מחלקת ה-ViewModel שמנהלת את הנתונים והפעולות (ViewModel-First)?
- למעשה, קיימות שתי גישות עיקריות, ולכל אחת מהן יתרונות ומלכודות.

1.2 ניתוח השוואתי: שתי הגישות

גישה א': View-First (להתחיל מהמסך)

בגישה זו, התלמיד מתחיל בעיצוב ה-XAML: מניח תוויות, שדות קלט וכפתורים, לעיתים עם טקסטים קבועים (Hardcoded) כדי לראות איך המסך נראה בעיניים.

• היתרונות:

- **משוב ויזואלי מיידי:** רואים מיד את התוצאה הגרפית על המסך.
- **הגדרת דרישות אינטואיטיבית:** תוך כדי הנחת הפקדים מבינים אילו שדות נחוצים.

• המלכודות לתלמיד:

- **הפיתוי לשבור MVVM:** קל מאוד להתפתות וללחוץ דאבל-קליק על כפתור כדי לכתוב אירוע `Button_Clicked` בקוד האחורי (Code-Behind), במקום לעבוד עם פקודות (`Command`).
- **Bindings "באוויר":** כותבים `{Binding SaveCommand}` במסך לפני שהפקודה קיימת בקוד, ללא השלמה אוטומטית (IntelliSense), וקל מאוד לטעות באיות של שמות המאפיינים.

גישה ב': ViewModel-First (להתחיל מהלוגיקה והמצב)

בגישה זו, לאחר שהמודל קיים, שואלים שתי שאלות מנחות:

1. **מהו המצב (State) של המסך?** מה המשתמש צריך לראות בכל רגע נתון? (למשל: סרטון נוכחי, האם יש טעינה, טקסט מיקום, ערכי שדות הקלט).
2. **אילו פעולות (Commands) המשתמש יכול לבצע?** (למשל: דפדוף קדימה, דפדוף אחורה, שמירה, איפוס).

• היתרונות (הגישה המקצועית):

- **בדיקותיות מלאה (Testability):** ניתן לבדוק שהלוגיקה עובדת במלואה עוד לפני שנגענו ב-XAML!
- **ה-XAML הופך ל"הלבשה" בלבד:** כאשר ניגשים ל-XAML, כל המאפיינים והפקודות כבר קיימים, פועלים ומוכרים על ידי מנגנון ה-IntelliSense וה-Compiled Bindings.

- **החיסרון לתלמיד:** דורש יכולת הפשטה לדמיין את התנהגות המסך מבלי לראות אותו עדיין בעיניים.

1.3 "שביל הזהב" המומלץ לתלמידים (The 4-Step Recipe)

כדי לחבר בין האינטואיציה הוויזואלית לבין משמעת ארכיטקטונית נכונה, מומלץ לעבוד לפי תהליך 4 השלבים:

TEXT

ב-2 דקות (Wireframe) שלב 1: שרטוט מהיר על דף נייר
↓
(Service) ושירות הנתונים (Model) שלב 2: מודל הנתונים
↓
(State & Commands-תרגום השרטוט ל-ViewModel-שלב 3: מחשבת ה
↓
(מובטחים Bindings חיבור) ב-XAML View-שלב 4: הלבשת ה

1. **שלב 1 – שרטוט מהיר ב-2 דקות על דף (Wireframe):** מציירים מלבן פשוט של סמארטפון. מסמנים איפה יושבים הכפתורים ואיפה שדות הקלט. השרטוט הזה מעניק מענה מיידי לשאלה: מה המשתמש רואה ומה הוא עושה?
2. **שלב 2 – מודל ושירות:** מגדירים את ישות הנתונים (`VideoItem`) ואת הממשק שמספק אותם (`IVideoService`).
3. **שלב 3 – כתיבת ה-ViewModel:** גוזרים ישירות מהשרטוט:
 - כל טקסט או שדה שמשתנה על המסך → הופך ל-`Property` (עם `OnPropertyChanged`).
 - כל כפתור שהמשתמש לוחץ עליו → הופך ל-`Command` (עם תנאי `CanExecute`).
4. **שלב 4 – הלבשת ה-XAML:** פותחים את ה-XAML, מניחים את הפקדים, ומבצעים `{Binding}` למאפיינים ולפקודות שכבר נבדקו ונמצאים ב-ViewModel.

פרק 2: יישום הלכה למעשה באפליקציית PlayListSample

2.1 המטרה הפדגוגית של השלב הנוכחי

לפני שלומדים להציג רשימות ואוספים דינמיים (`ObservableCollection` ו-`CollectionView`), חיוני לחזור ולשלוט בצורה מושלמת באבני היסוד של תבנית ה-MVVM:

- הפרדת אחריות (Separation of Concerns):** המסך (View) לא יודע מהיכן מגיעים הנתונים; ה-ViewModel לא ניגש לפקדי ה-UI ישירות; וה-Service מנהל את שליפת ושמירת הנתונים.
- תצוגת פריט בודד והצורך בדפדוף:** כרגע, כאשר אין לנו עדיין רכיב אוסף על המסך, הדרך היחידה לעבור בין סרטונים היא **דפדוף ידני** (כפתורי "הבא" ו"הקודם"). התנסות זו ממחישה לתלמיד מדוע בהמשך נזדקק ל-`CollectionView` שציג את כל הפריטים יחד.
- תשתית א-סינכרונית נקייה:** שימוש ב-`interface` עם מתודות `Task` המאפשר בעתיד להחליף את ה-Mockup הפנימי ב-SQLite אמיתי מבלי לשנות שורת קוד אחת ב-ViewModel!

2.2 מבנה השכבות בפרויקט

TEXT

```
PlayListSample/
├── Models/
│   └── VideoItem.cs          # מודל הנתונים הבסיסי (POCO)
├── Services/
│   ├── IVideoService.cs     # ממשק הפעולות (חוזה א-סינכרוני)
│   └── MockVideoService.cs  # מימוש סטטי בזיכרון (Mockup)
├── ViewModels/
│   ├── BaseViewModel.cs     # מימוש INotifyPropertyChanged
│   └── MainViewModel.cs     # ניהול מצב המסך, דפדוף ופקודות
├── MainPage.xaml             # (מורכבים Styles בלבד, ללא Bindings) XAML-עיצוב המסך ב
├── MainPage.xaml.cs          # באופן ישיר ושקוף BindingContext-הצמדת ה
├── App.xaml / App.xaml.cs    # (ללא Shell) MainPage-נקודת כניסה ישירה ל
└── MauiProgram.cs            # (ללא DI Container) אתחול האפליקציה
```

2.3 ניתוח מעמיק של מרכיבי הקוד

א. המודל (`VideoItem.cs`)

המודל הוא מחלקת #C פשוטה (POCO) המייצגת סרטון בודד:

- Id**: מזהה ייחודי (מכין את הקרקע למפתח ראשי ב-SQLite).
- Title**, **Url**, **ChannelName**: שדות טקסטואליים.

- `DurationMinutes`: משך הזמן בדקות.
- `IsWatched`: מצוין האם הסרטון נצפה.
- מאפייני עזר לקריאה בלבד: `FormattedDuration` ו-`WatchedStatusText` המאפשרים הצגה נקייה ב-XAML ללא צורך בממירי ערכים (ValueConverters).

ב. ממשק השירות (`IVideoService.cs`) ושירות המוק (`MockVideoService.cs`)

- **מדוע יצרנו `interface`?** ה-`ViewModel` תלוי אך ורק בממשק `IVideoService`. הוא לא יודע אם הנתונים מגיעים מרשימה בזיכרון, מקובץ מקומי, ממסד נתונים SQLite או משרת ענן.
- **מדוע כל הפעולות מחזירות `Task`?**

CSHARP

```
Task<IReadOnlyList<VideoItem>> GetAllVideosAsync();
Task<bool> AddVideoAsync(VideoItem video);
```

אף שכרגע הנתונים נמצאים ברשימה סטטית בזיכרון, הגדרת המתודות כ-`async` ושימוש ב-`await Task.Delay(...)` מרגילים את התלמיד לעבודה נכונה עם תהליכים א-סינכרוניים, ומבטיחים שכאשר נעבור ל-SQLite – חתימות המתודות לא ישתנו כלל.

ג. מחלקת הבסיס (`BaseViewModel.cs`)

מממשת את `INotifyPropertyChanged`:

- ברגע שערך של מאפיין משתנה, אנו מפעילים את `OnPropertyChanged()`.
- המאפיין `IsLoading` משמש לבקרת מחוון הטעינה (`ActivityIndicator`).
- המאפיין הנגזר `IsNotLoading` מאפשר לחבר תנאי תצוגה והפעלה בצורה אלגנטית ב-XAML.

ד. ה-`ViewModel` הראשי (`MainViewModel.cs`)

מנהל את כל מה שקורה במסך:

1. **דפדוף (Browsing):** מחזיק רשימה פנימית ואינדקס `_currentIndex`.
2. **פקודות עם `CanExecute`:**
 - `NextCommand`: פעיל רק אם `_currentIndex < _videos.Count - 1`.
 - `PreviousCommand`: פעיל רק אם `_currentIndex > 0`.
 - כאשר עוברים סרטון, קוראים ל-`ChangeCanExecute()` על הפקודות, וכך כפתור "הקודם" נהיה אפור/מנוטרל בסרטון הראשון, וכפתור "הבא" מנוטרל בסרטון האחרון!
3. **טופס הזנה:** מחזיק מאפיינים כמו `InputTitle`, `InputChannelName`, `InputUrl` המחוברים ב-`Mode=TwoWay` לשדות ה-`Entry`.
4. **שמירה:** פקודת `SaveCommand` מבצעת ולידציה בסיסית, מוסיפה ל-`Service`, מעדכנת את הרשימה ומקפיצה את התצוגה לסרטון החדש שנוסף.

בשלב הלימודי הזה, איננו מסתירים את יצירת האובייקטים מאחורי מכולת הזרקת תלויות (DI Container):

CSHARP

```
public MainPage()
{
    InitializeComponent();
    BindingContext = new MainViewModel(new MockVideoService());
}
```

התלמיד רואה בשורה אחת ברורה כיצד שירות ה-Mock מועבר ל-ViewModel, וכיצד ה-ViewModel מוגדר כ- `BindingContext` של הדף.

2.4 שאלות חשיבה ואתגרים לתלמיד

1. **שאלה מתודולוגית:** נניח שבשרטוט המסך שלך מופיע כפתור "נקה טופס". לפי מתודולוגיית ה-ViewModel-First, איזה רכיב תגדיר ב-ViewModel ואיזה מאפיינים תאפס בתוכו?
2. **שאלה אדריכלית:** מדוע עדיף לחבר כפתור לפקודת `ICommand` ב-ViewModel עם `CanExecute`, במקום לכתוב אירוע `Clicked` ב-Code-Behind עם `if (index > 0) button.IsEnabled = ...`?
3. **שאלת הפרדת שכבות:** אם מחר נרצה להחליף את `MockVideoService` במסד נתונים של SQLite, אילו קבצים בפרויקט נצטרך לשנות, ואילו קבצים יישארו ללא שינוי?
4. **גשר לקראת השלב הבא:** מה החיסרון העיקרי בתצוגת פריט בודד עם דפדוף ("הבא"/"הקודם"), ומדוע הצגת רשימה מלאה (`ObservableCollection`) תהיה נוחה בהרבה למשתמש?

פרק 3: שיטת הבלוקים המודולרית (The Block Strategy) –

עיצוב ממשק המשתמש במסך MainPage

תאריך: 2026-09-07

בפרק זה נלמד כיצד להרכיב את ממשק המשתמש ב-XAML בצורה מודולרית והדרגתית לפי שיטת הבלוקים המודולרית (*The Block Strategy*). במקום לכתוב קובץ XAML ארוך ומאיים בבת אחת, אנו מפרקים את המסך ליחידות חזותיות ולוגיות עצמאיות.

3.1 מפת הבלוקים של המסך (Visual Block Architecture)

מסך ה- `MainPage` מורכב משני בלוקים עיקריים בנוסף לבאנר הראשי:

1. **באנר עליון (Hero Section):** כותרת צבעונית המציגה את מטרת המסך.

2. **בלוק 1 – דפדוף וצפייה בסרטון בודד (Single-Item Viewer & Pagination):** סרגל כפתורי ניווט מוגנים ותצוגת כל פרטי הסרטון הנוכחי.

3. **בלוק 2 – טופס הוספת סרטון (Two-Way Form Input):** שדות קלט עם סוגי מקלדות מותאמים, כפתור שמירה וכפתור ניקוי טופס.



09:41

ניהול פלייליסט - חזרה על MVVM

דפדוף, פקודות CanExecute וטופס הזנה Two-Way

בלוק 1: דפדוף בסרטונים (ITEM VIEWER)

דפדוף בסרטונים (תצוגת פריט בודד) 

→ הבא

סרטון 2 מתוך 4

← הקודם

מבוא ל-Data Binding ב-MAUI

כותרת:

DotNetIL

ערוץ:

24 דק'

משך:

youtu.be/sample-binding

כתובת:

נצפה ✓

סטטוס:

שנה סטטוס צפייה 

בלוק 2: טופס הזנה (TWO-WAY BINDING)

+ הוספת סרטון חדש (טופס קלט)

כותרת הסרטון:

פקודות ICommand ו-CanExecute

שם הערוץ:

Microsoft Developers

משך בדקות:

18

כתובת אינטרנט (URL):

<https://youtube.com/watch?v=demo2>

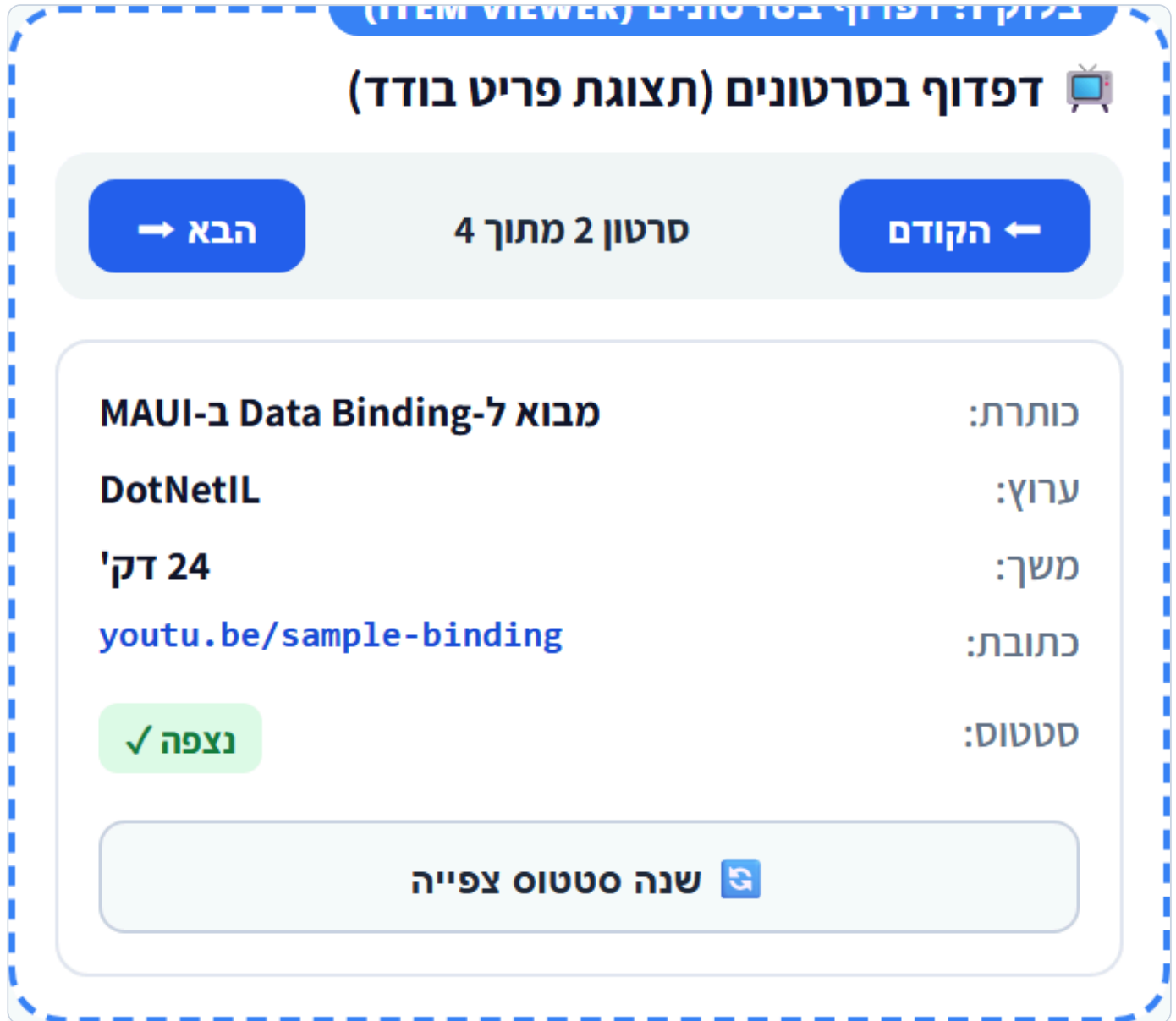
נקה טופס 

שמור סרטון 

הסרטון 'פקודות ICommand' נוסף בהצלחה לרשימה!

3.2 פירוק הבלוקים והרכבת ה-XAML

בלוק 1: דפדוף ותצוגת פריט בודד



בלוק זה עטוף ב- `Border` בעל פינות מעוגלות (`RoundRectangle 8`) ומסגרת בהירה. הוא כולל:

- **סרגל דפדוף עליון:** גריד בעל 3 עמודות (`Auto, *, Auto`) המאכלס את כפתור "הקודם", תווית מיקום מחושבת (`CurrentPositionText`), וכפתור "הבא".
- **תנאי:** `CanExecute`: הכפתורים מנוטרלים אוטומטית כשהמשתמש מגיע לתחילת או סוף הרשימה, ובכך נמנעות קריסות של `IndexOutOfRangeException`!
- **תצוגת פרטי הסרטון:** תוויות המציגות את הכותרת, שם הערוץ, משך הזמן והכתובת.
- **כפתור שינוי סטטוס צפייה:** מחובר ישירות לפקודת `ToggleWatchedCommand`.

בלוק 2: טופס הוספת סרטון (Two-Way Binding)

+

הוספת סרטון חדש (טופס קלט)

כותרת הסרטון:

פקודות ICommand ו-CanExecute

שם הערוץ:

Microsoft Developers

משך בדקות:

18

כתובת אינטרנט (URL):

https://youtube.com/watch?v=demo2

נקה טופס 

שמור סרטון 

הסרטון 'ICommand' נוסף בהצלחה לרשימה!

בלוק זה מאפשר למשתמש להזין פרטי סרטון חדש:

- `Mode=TwoWay`: כל הקלדה של המשתמש בתיבות ה-`Entry` מעדכנת את המאפיינים המתאימים ב-`MainViewModel`.
- **מקלדת ייעודית:** שימוש ב-`Keyboard="Numeric"` עבור שדה משך הזמן בדקות.
- **גרید פעולות תחתון:** כפתור "שמור סרטון" (בצבע ירוק) וכפתור "נקה טופס".

3.3 פינות העמקה והבנה מושגית (Deep-Dive Callouts)

מהו Two-Way Data Binding ומדוע הוא קריטי בטפסים?

בעוד שבתוויות טקסט רגילות (`Label`) הנתונים זורמים רק מהקוד למסך (`One-Way`), בשדות קלט (`Entry`) אנו זקוקים לזרימה דו-כיוונית (`Two-Way`). כאשר המשתמש מקליד תו בתיבת הטקסט, המאפיין ב-`ViewModel` מתעדכן בו-ברגע. כאשר לוחצים על "שמור", המידע כבר מוכן בזיכרון של ה-`ViewModel`!

הגנה הרמטית על כפתורי הניווט באמצעות `CanExecute`

במקום לכתוב קוד אירוע מסורבל בקובץ ה-`Behind-Code` (`button.IsEnabled = if (_currentIndex == 0) true else false`), תבנית ה-`MVVM` מאפשרת להגדיר תנאי ביצוע ישירות בתוך הפקודה:

CSHARP

```
PreviousCommand = new Command(ExecutePrevious, CanExecutePrevious);  
private bool CanExecutePrevious() => IsNotLoading && _videos.Count > 0 &&  
_currentIndex > 0;
```

בכל שינוי אינדקס, הקריאה ל-`ChangeCanExecute()` גורמת ל-`MAUI` לרענן את מצב הכפתור ללא שורת קוד אחת של ממשק!

3.4 שאלות חזרה, הבנה ואתגר חשיבה

דילמת תכנון: דפדוף יחיד מול רשימת כרטיסים מלאה 🚖

- **דפדוף פריט בודד (Single-Item Pagination):** מאפשר התמקדות מלאה בתוכן, חוסך זיכרון ומעבד, ומצוין עבור שאלוני בחינה, כרטיסיות לימוד, או ממשקי נגנים.
- **רשימה מלאה (CollectionView):** מאפשרת למשתמש לסקור במבט אחד את כל הפריטים, לגלול במהירות ולחפש.

שאלות הבנה מעמיקות 💡

1. מדוע חיוני לקרוא ל-`ChangeCanExecute()` על הפקודות בתוך `RefreshCommands()`?
2. מה היה קורה אם היינו מוחקים את הגדרת `Mode=TwoWay` בשדות ה-`Entry` של הטופס?

נספח א' – סדרת מדריכי ממשק משתמש בשיטת הבלוקים המודולרית

חלק זה מרכז את מדריכי הממשק המודולריים של האפליקציה:

- [מדריך פיתוח בשלבים למתחילים: PlaylistSample צעד-אחר-צעד](#) – מדריך פדגוגי מקיף מאפס: סקיצה, מודל, ViewModel לדפדוף ללא שירות, XAML ממוקד, הוספת שירות ו-Mockup, טופס הוספה Two-Way, שאלות הבנה בסוף כל פרק, ונספח מלא על בדיקות יחידה xUnit.
- [מדריך פיתוח מסך ראשי: MainPage](#) – פירוק מודולרי ל-2 בלוקים (דפדוף בסרטון בודד וטופס הזנה Two-Way), פינות העמקה טכנולוגיות, ואתגר מעשי מודרך.

נספח ב' – מקורות השראה, תיעוד וביבליוגרפיה

ספר למידה זה ומדריכי הממשק שבו פותחו בהשראת מתודולוגיית ההוראה המודולרית **The Block Strategy** של **Leomaris Reyes** (Microsoft MVP, בלוגרית Telerik ויוצרת AskXammy.com), המנגישה עיצובי ממשק מובייל מורכבים ב-.NET MAUI. בצורה הדרגתית ומובנית.

תיעוד רשמי ומקורות להעמקה:

- [Microsoft Learn \(.NET MAUI\)](#): תיעוד רשמי של ארכיטקטורת האפליקציה, מחזור החיים ובקרי הממשק.
- [Microsoft Learn: Data Binding & MVVM](#): דפוסי חיבור נתונים, פקודות ICommand ו-Compiled Bindings.
- [Microsoft Learn: The Command Interface](#): שימוש ב-ICommand ותנאי CanExecute להגנה על פקודות.
- [Microsoft Learn: Data Binding Modes](#): קישור נתונים One-Way מול Two-Way ב-.NET MAUI.

פרק הרחבה – מדריך פיתוח מפורט: MainPage (שיטת הבלוקים המודולרית)

מדריך פיתוח מסך ראשי: MainPage – ניהול פלייליסט ודפדוף ב-MVVM

שיטת הבלוקים המודולרית (The Block Strategy) לבניית ממשק משתמש ב-.NET MAUI

1. מבוא והשראה עיצובית: בונים מסך ניהול תוכן ממוקד ואינטראקטיבי

אי פעם שאלתם את עצמכם איך אפליקציות מציגות פריט בודד בצורה ממוקדת וברורה – כמו נגן שירים שמאפשר לדפדף בין רצועות, או כרטיסיית למידה שבה מתמקדים בכל פעם במושג אחד בלבד? לצד רשימות ארוכות, **תצוגת פריט בודד עם דפדוף**

(Single-Item Pagination) היא אחד מדפוסי הממשק החשובים ביותר בפיתוח אפליקציות מובייל: היא מונעת עומס קוגניטיבי ומאפשרת למשתמש להתרכז בתוכן הנוכחי.

במדריך זה נבנה צעד-אחר-צעד את מסך ה- `MainPage` של פרויקט `PlaylistSample`. נלמד כיצד לשלב במסך אחד שני צרכים מרכזיים:

1. **דפדוף חלק בסרטונים** עם כפתורי קדימה/אחורה המוגנים בלוגיקת `CanExecute`.
2. **טופס הזנה עשיר** המעדכן את הנתונים בזמן אמת בטכניקת `Two-Way Data Binding`.

מפת הבלוקים של המסך (Visual Block Architecture)

להלן מפת היעד החזותית של המסך, כפי שחולקה לבלוקים פונקציונליים ברורים:



09:41

ניהול פלייליסט - חזרה על MVVM

דפדוף, פקודות CanExecute וטופס הזנה Two-Way

בלוק 1: דפדוף בסרטונים (ITEM VIEWER)

דפדוף בסרטונים (תצוגת פריט בודד) 

→ הבא

סרטון 2 מתוך 4

← הקודם

מבוא ל-Data Binding ב-MAUI

כותרת:

DotNetIL

ערוץ:

24 דק'

משך:

youtu.be/sample-binding

כתובת:

נצפה ✓

סטטוס:

שנה סטטוס צפייה 

בלוק 2: טופס הזנה (TWO-WAY BINDING)

+ הוספת סרטון חדש (טופס קלט)

כותרת הסרטון:

פקודות ICommand ו-CanExecute

שם הערוץ:

Microsoft Developers

משך בדקות:

18

כתובת אינטרנט (URL):

https://youtube.com/watch?v=demo2

נקה טופס

שמור סרטון

הסרטון 'פקודות ICommand' נוסף בהצלחה לרשימה!

2. איך המדריך הזה בנוי? (The Meta-Frame) 🗺️

כדי שתוכלו להבין כל שורת קוד ולשלוט בממשק ב-100%, נפעל לפי 3 עקרונות פדגוגיים מנחים:

- **דיאגרמה חזותית מפורקת (Visual Diagram):** חילקנו את המסך לשני בלוקים צבעוניים עיקריים בנוסף לבאנר הראשי. בכל שלב נתמקד בבלוק אחד בלבד.
- **קוד מודולרי עם נקודות עוגן (Modular Code with Placeholders):** נתחיל עם שלד הגריד והערות מקום (כאן <!--> יבוא הבלוק הבא), ונרכיב כל רכיב בנפרד.
- **התקדמות הדרגתית מלווה בצילומי מסך ממוקדים:** לכל בלוק מוצגת תמונת תוצאה מתוך ההדמיה, לצד פינות העמקה (Deep-Dive Callouts) על טכניקות מפתח.

3. הכנת סביבת העבודה ומרחבי שמות (Namespaces) 🛠️

לפני שמתחילים בעיצוב ה-XAML, נוודא שראש הקובץ `MainPage.xaml` כולל את מרחבי השמות הדרושים לקישור ה-ViewModel והמודלים, וכן את הגדרת כיוון המסך מימין-לשמאל:

```
XML
```

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
  xmlns:vm="clr-namespace:PlaylistSample.ViewModels"
  x:Class="PlaylistSample.Views.MainPage"
  x:DataType="vm:MainViewModel"
  Title="{Binding Title}"
  FlowDirection="RightToLeft">
```

⏪ — כאן יבוא שלד הגריד הראשי

</ContentPage>

💡 **מדוע הגדרנו** `x:DataType="vm:MainViewModel"` ?

זהו מנגנון **Compiled Bindings** של .NET MAUI. הוא מאפשר לקומפילר לבדוק בזמן הידור שכל שדה ופקודה קיימים באמת ב-ViewModel, מונע שגיאות כתיב חבויות, ומשפר את ביצועי האפליקציה בריצה!

4. אסטרטגיית הבלוקים (The Block Strategy) 📐

⚠️ **הנחיית כתיבת קוד (XAML Line-Breaking Standard):** בספרי תכנות מקצועיים לעולם אין לכתוב תגיות XAML ארוכות העולות על 65-70 תווים בשורה אחת. בכל תגית המכילה 2 תכונות ומעלה — יש לרדת שורה עבור כל תכונה (*Attribute-per-line* *indent*). זה מונע חיתוך טקסט ב-PDF ומבטיח קוד קריא ונקי!

שלב 4.0: שלד הגריד הראשי ומחווון הטעינה (Layout Backbone & Loading)

עמוד השדרה של המסך עטוף ב- `ScrollView` המאפשר גלילה חלקה בכל גודל מסך:

XML

```
<ScrollView>
  <VerticalStackLayout Padding="20"
    Spacing="16">

    ⏪ — כותרת ראשית ומחווון טעינה
    <Label Text="{Binding Title}"
      FontSize="22"
      FontAttributes="Bold"
      HorizontalOptions="Center" />

    <ActivityIndicator IsRunning="{Binding IsLoading}"
      IsVisible="{Binding IsLoading}"
      HorizontalOptions="Center" />
```

<-- בלוק 1: דפדוף ותצוגת סרטון בודד -->
<-- בלוק 2: טופס הוספת סרטון חדש -->

```
</VerticalStackLayout>  
</ScrollView>
```

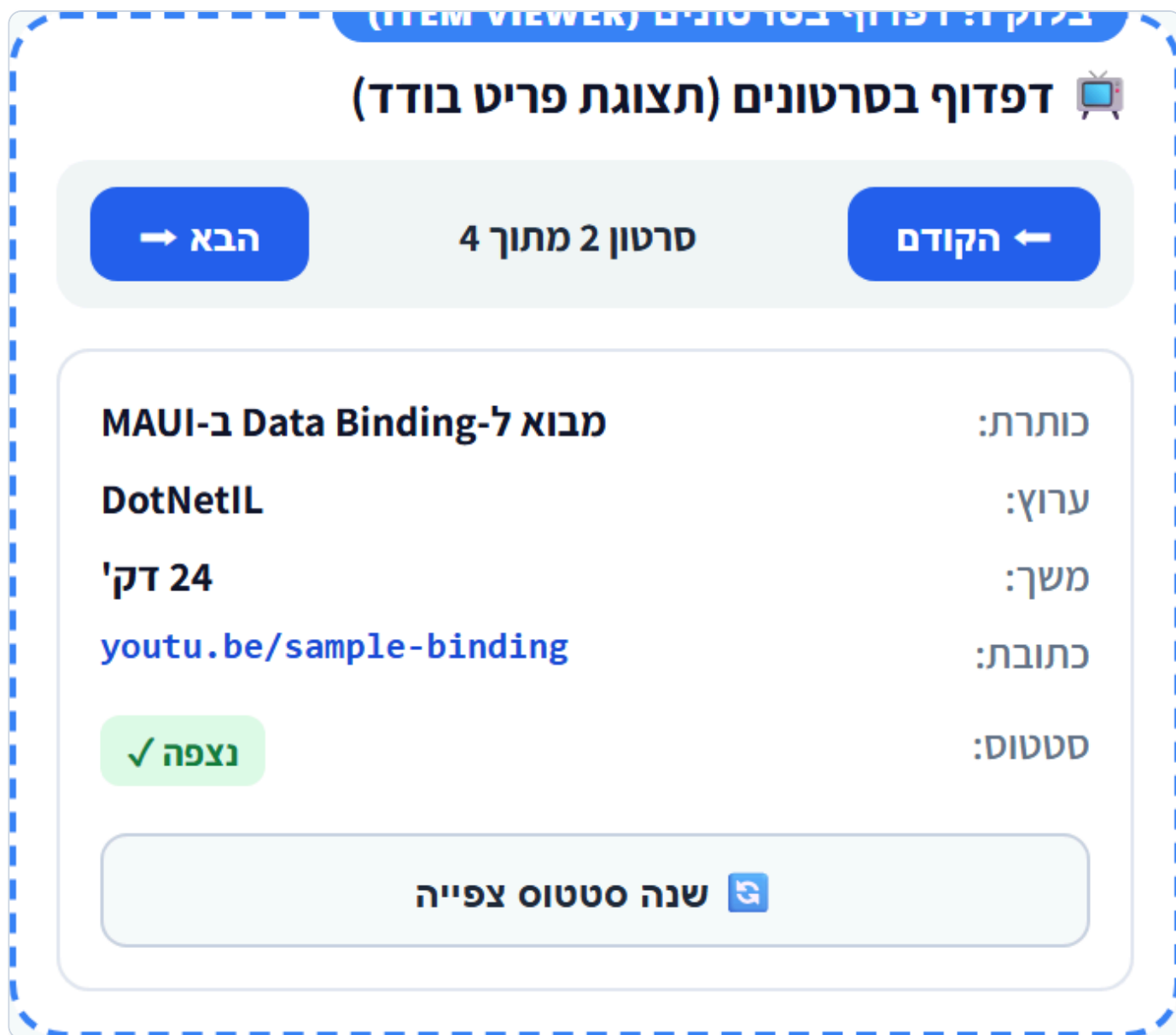
שלב 4.1: כותרת עליונה ובאנר ראשי (Hero Banner)

ניהול פלייליסט - חזרה על MVVM 
דפדוף, פקודות CanExecute וטופס הזנה Two-Way

כרטיס כותרת מעוצב בגווני כחול המעניק חויית פתיחה מקצועית:

XML

```
<Border StrokeShape="RoundRectangle 12"  
    BackgroundColor="#2563EB"  
    Padding="16"  
    StrokeThickness="0">  
    <VerticalStackLayout Spacing="4">  
        <Label Text="ניהול פלייליסט - חזרה על MVVM"  
            FontSize="19"  
            FontAttributes="Bold"  
            TextColor="White"  
            HorizontalOptions="Center" />  
        <Label Text="Two-Way וטופס הזנה CanExecute דפדוף, פקודות"  
            FontSize="12"  
            TextColor="#DBEAFE"  
            HorizontalOptions="Center" />  
    </VerticalStackLayout>  
</Border>
```



בלוק זה מציג סרטון אחד בכל רגע נתון מתוך הרשימה, ומאפשר לנווט בין סרטונים בעזרת כפתורי "הקודם" ו"הבא" לצד שינוי סטטוס הצפייה:

XML

```
<!-- חלק 1: דפדוף ותצוגת פריט בודד מתוך השירות -->
<Border StrokeShape="RoundRectangle 8"
    Padding="14"
    BackgroundColor="White"
    Stroke="#E2E8F0">
    <VerticalStackLayout Spacing="10">

        <Label Text="דפדוף בסרטונים (תצוגת פריט בודד)"
            FontSize="16"
            FontAttributes="Bold"
            TextColor="#0F172A" />
```

```

<!-- סרגל דפדוף (Next / Previous) -->
<Grid ColumnDefinitions="Auto, *, Auto">
    <Button Grid.Column="0"
        Text="← הקודם"
        Command="{Binding PreviousCommand}"
        BackgroundColor="#2563EB"
        TextColor="White" />

    <Label Grid.Column="1"
        Text="{Binding CurrentPositionText}"
        FontAttributes="Bold"
        FontSize="12"
        HorizontalOptions="Center"
        VerticalOptions="Center" />

    <Button Grid.Column="2"
        Text="→ הבא"
        Command="{Binding NextCommand}"
        BackgroundColor="#2563EB"
        TextColor="White" />
</Grid>

<BoxView HeightRequest="1"
    Color="#E2E8F0"
    Margin="0,4" />

<!-- פרטי הפריט הנוכחי המוצג על המסך -->
<Label Text="{Binding CurrentVideo.Title, StringFormat='{0} : כותרת:}'"
    FontSize="15"
    FontAttributes="Bold" />

<Label Text="{Binding CurrentVideo.ChannelName, StringFormat='{0} : ערוץ:}'"
    FontSize="13"
    TextColor="#475569" />

<Label Text="{Binding CurrentVideo.FormattedDuration, StringFormat='{0} : משך:}'"
    FontSize="13"
    TextColor="#475569" />

<Label Text="{Binding CurrentVideo.Url, StringFormat='{0} : כתובת אינטרנט:}'"
    FontSize="12"
    TextColor="#1D4ED8"
    LineBreakMode="TailTruncation" />

<Label Text="{Binding CurrentVideo.WatchedStatusText, StringFormat='{0} : סטטוס:}'"
    FontSize="13"
    FontAttributes="Bold" />

<Button Text="🔄 שנה סטטוס צפייה"
    Command="{Binding ToggleWatchedCommand}"
    BackgroundColor="#F8FAFC"
    TextColor="#1E293B"
    BorderColor="#CBD5E1"
    BorderWidth="1"
    Margin="0,6,0,0" />

</VerticalStackLayout>
</Border>

```

כותרת הסרטון:

שם הערוץ:

משך בדקות:

18

כתובת אינטרנט (URL):

<https://youtube.com/watch?v=demo2>

נקה טופס

שמור סרטון

הסרטון 'פקודות ICommand' נוסף בהצלחה לרשימה!

בלוק זה מאפשר למשתמש להקליד נתוני סרטון חדש. הקישור הדו-כיווני (`Mode=TwoWay`) מעדכן מיידית את שדות ה- `ViewModel`, וכפתור השמירה מוסיף את הסרטון ועובר אליו אוטומטית:

```
<!-- חלק 2: טופס הזנה של פריט בודד -->
<Border StrokeShape="RoundRectangle 8"
```

```

        Padding="14"
        BackgroundColor="White"
        Stroke="#E2E8F0">
<VerticalStackLayout Spacing="8">

    <Label Text="+ (טופס קלט)"
        FontSize="16"
        FontAttributes="Bold"
        TextColor="#0F172A" />

    <Label Text="כותרת הסרטון:"
        FontSize="12"
        TextColor="#475569" />
    <Entry Placeholder="הזן כותרת"
        Text="{Binding InputTitle, Mode=TwoWay}" />

    <Label Text="שם הערוץ:"
        FontSize="12"
        TextColor="#475569" />
    <Entry Placeholder="הזן ערוץ"
        Text="{Binding InputChannelName, Mode=TwoWay}" />

    <Label Text="משך בדקות:"
        FontSize="12"
        TextColor="#475569" />
    <Entry Placeholder="למשל 15"
        Keyboard="Numeric"
        Text="{Binding InputDurationText, Mode=TwoWay}" />

    <Label Text="כתובת אינטרנט (URL):"
        FontSize="12"
        TextColor="#475569" />
    <Entry Placeholder="https:// ..."
        Text="{Binding InputUrl, Mode=TwoWay}" />

    <Grid ColumnDefinitions="*, *"
        ColumnSpacing="10"
        Margin="0,8,0,0">
        <Button Grid.Column="0"
            Text="שמור סרטון"
            Command="{Binding SaveCommand}"
            BackgroundColor="#10B981"
            TextColor="White" />

        <Button Grid.Column="1"
            Text="נקה טופס"
            Command="{Binding ClearFormCommand}"
            BackgroundColor="#F1F5F9"
            TextColor="#475569" />
    </Grid>

    <Label Text="{Binding StatusMessage}"
        FontSize="12"
        TextColor="#4338CA"
        HorizontalOptions="Center" />

</VerticalStackLayout>
</Border>

```

5. פינות העמקה והבנה מושגית (Deep-Dive Callouts)

1. מה ההבדל בין קישור חד-כיווני (One-Way) לדו-כיווני (Two-Way)?

- בקישור **One-Way** (ברירת המחדל ברכיבי **Label**): הנתונים זורמים רק מה-ViewModel אל המסך. שינוי ב-C# מעדכן את התצוגה.
- בקישור **Two-Way** (בשדות **Entry** בטופס): הנתונים זורמים בשני הכיוונים! כל תו שהמשתמש מקליד בתיבת הטקסט מוזרם אוטומטית אל התכונה המתאימה ב-ViewModel, כך שכשלווחצים על "שמור", הנתונים כבר מעודכנים ב-C# ללא צורך בקריאה ידנית של **entry.Text**!

תיעוד רשמי: [Microsoft Learn - Data Binding Modes](#)

2. כיצד **CanExecute** מגן על כפתורי הניווט מפני קריסות?

מה היה קורה אילו המשתמש היה לוחץ על "הקודם" כשהוא כבר בסרטון הראשון (**Index = 0**), או על "הבא" בסרטון האחרון? הקוד היה מנסה לגשת לאינדקס **-1** או **Count** ומקפיץ מיד חריגת **ArgumentOutOfRangeException** שהייתה קורסת את האפליקציה! בזכות הגדרת **CanExecuteNext** ו-**CanExecutePrevious** ב-**MainViewModel**:

CSHARP

```
private bool CanExecutePrevious() => IsNotLoading && _videos.Count > 0 &&
_currentIndex > 0;
private bool CanExecuteNext() => IsNotLoading && _videos.Count > 0 && _currentIndex <
_videos.Count - 1;
```

מערכת .NET MAUI מנטרלת את הכפתור (אפור) ברגע שמגיעים לקצה הרשימה, וחוסמת לחיצות שגויות באופן הרמטי!

תיעוד רשמי: [Microsoft Learn - The Command Interface](#)

3. למה להשתמש ב-**ActivityIndicator** ולא ב-**Page.IsBusy**?

בעבר ב-Xamarin השתמשו לעיתים במאפיין של הדף. בארכיטקטורת MVVM מודרנית, הדף אינו אמור לנהל מצבי מערכת. ה-ViewModel מחזיק את המשתנה הבוליאני **IsLoading**, ומעדכן את הממשק דרך **INotifyPropertyChanged**. כך, הלוגיקה ניתנת לבדיקה מלאה בבדיקות יחידה (Unit Tests) ללא תלות בממשק גרפי!

6. ארכיטקטורת MVVM וחיבור הנתונים (State & Commands)

כדי לחבר את הממשק בצורה נקייה, ה- `MainViewModel` מרכז את ניהול המצב והפעולות:

1. ניהול הפריט המוצג וטקסט המיקום

CSHARP

```
public VideoItem? CurrentVideo
{
    get => _currentVideo;
    private set
    {
        if (_currentVideo != value)
        {
            _currentVideo = value;
            OnPropertyChanged();
        }
    }
}

public string CurrentPositionText
{
    get => _currentPositionText;
    private set
    {
        if (_currentPositionText != value)
        {
            _currentPositionText = value;
            OnPropertyChanged();
        }
    }
}
```

2. עקרון פיתוח מקצועי: רענון פקודות (`RefreshCommands`)

בכל פעם שהאינדקס משתנה או שמתווסף סרטון חדש, אנו מפעילים רענון לכל הפקודות:

CSHARP

```
private void RefreshCommands()
{
    ((Command)NextCommand).ChangeCanExecute();
    ((Command)PreviousCommand).ChangeCanExecute();
    ((Command)SaveCommand).ChangeCanExecute();
    ((Command)ToggleWatchedCommand).ChangeCanExecute();
    ((Command)ClearFormCommand).ChangeCanExecute();
}
```

קריאה זו מאותתת ל-XAML לבדוק מחדש את תנאי ה- `CanExecute`, ומשנה את מצב הכפתורים (זמינים או מושבתים) מיד!

? 7. שאלות חזרה, הבנה ואתגר חשיבה

⚖️ חלק א': דילמת ארכיטקטורה ועיצוב (Architectural Dilemma)

הדילמה: תצוגת דפדוף בודד (Single-Item Pagination) לעומת רשימה נגללת (CollectionView)?

בפרויקט הנוכחי בחרנו להציג סרטון אחד בכל פעם עם כפתורי דפדוף, בעוד שבפרויקטים אחרים משתמשים ברשימת כרטיסים נגללת (`CollectionView`).

- **יתרונות תצוגת פריט בודד:** מיקוד מקסימלי של המשתמש, חיסכון בזיכרון (המסך אינו צריך לצייר עשרות פריטים בו-זמנית), ומניעת בלבול בעת עבודה על פריט ספציפי (כגון שאלון, כרטיסיית למידה, או נגן).
- **יתרונות רשימה נגללת:** סקירה מהירה של הרבה פריטים בו-זמנית, גלילה רציפה, וחיפוש ויזואלי מהיר.

שיקול החלטה למפתח: בחרו בתצוגת פריט בודד כאשר המשתמש צריך להתרכז בפעולה על אובייקט אחד (קריאה, עריכה, האזנה). בחרו ב- `CollectionView` כאשר המטרה היא סקירה ובחירה מתוך קטלוג.

💬 חלק ב': שאלות הבנה מעמיקות עם רמזי הכוונה

1. מה יקרה אם נמחק את הקריאה ל- `ChangeCanExecute()` בתוך מתודת `RefreshCommands()`?

רמז לחשיבה: חשבו על כפתור "הבא" כשמגיעים לסרטון האחרון. האם הכפתור ישתנה לאפור מעצמו בלי שה- `ViewModel` יודיע ל-XAML לבדוק את התנאי שוב?

2. מדוע בשדות הטופס ציינו `Mode=TwoWay`, בעוד שבפרטי הסרטון הנוכחי הסתפקנו ב- `One-Way`?

רמז לחשיבה: מי יוזם את שינוי המידע בטופס? ומי יוזם את שינוי המידע בדפדוף בין סרטונים?

3. כיצד מנגנון ה- `ActivityIndicator` משתלב עם `IsLoading` כדי למנוע לחיצות כפולות בזמן שמירה?

רמז לחשיבה: שימו לב לתנאי הביצוע של פקודת השמירה: `IsNotLoading >= ()`. מה קורה לתנאי זה בזמן ש- `IsLoading = true`?

🔧 חלק ג': אתגר מעשי קצר לתלמיד (Mini-Challenge)

המשימה:

הוסיפו לסרגל הדפדוף בבלוק 1 שני כפתורים נוספים:

1. כפתור "ראשון" (`<<`): קופץ ישירות לסרטון הראשון ברשימה (`_currentIndex = 0`).
2. כפתור "אחרון" (`>>`): קופץ ישירות לסרטון האחרון ברשימה (`_currentIndex = _videos.Count - 1`).

1. שכבת ה-ViewModel (MainViewModel.cs):

- הגדירו שתי פקודות חדשות: `FirstCommand` ו-`LastCommand`.
- בבנאי חברו אותן למתודות ביצוע עם תנאי `CanExecute`:

CSHARP

```
FirstCommand = new Command(() => { _currentIndex = 0; UpdateCurrentVideo(); },
CanExecutePrevious);
LastCommand = new Command(() => { _currentIndex = _videos.Count - 1; UpdateCurrentVideo(); },
CanExecuteNext);
```

- הוסיפו את שתי הפקודות למתודת `RefreshCommands()`:

CSHARP

```
((Command)FirstCommand).ChangeCanExecute();
((Command>LastCommand).ChangeCanExecute();
```

2. שכבת ה-XAML (MainPage.xaml):

- שנו את הגריד של סרגל הדפדוף מ-`Auto, *, Auto` ל-`Auto, Auto, *, Auto, Auto`:

XML

```
<Grid ColumnDefinitions="Auto, Auto, *, Auto, Auto"
ColumnSpacing="6">
<Button Grid.Column="0"
Text="⏮"
Command="{Binding FirstCommand}" />

<Button Grid.Column="1"
Text="⏪ הקודם"
Command="{Binding PreviousCommand}" />

<Label Grid.Column="2"
Text="{Binding CurrentPositionText}"
HorizontalOptions="Center"
VerticalOptions="Center" />

<Button Grid.Column="3"
Text="הבא ⏩"
Command="{Binding NextCommand}" />

<Button Grid.Column="4"
Text="⏭" />
```

```
Command="{Binding LastCommand}" />
```

```
</Grid>
```

3. **אימות:** הריצו את האפליקציה, וודאו שכפתור "ראשון" מנוטרל כשאתם בסרטון 1, וכפתור "אחרון" מנוטרל כשמגיעים לסוף הרשימה!

נספח: מקורות השראה, תיעוד וביבליוגרפיה

מדריך זה נבנה בהשראת מתודולוגיית הפירוק המודולרי של ממשקי מובייל (The Block Strategy) כפי שפותחה ע"י **Leomaris Reyes** (Microsoft MVP, יוצרת AskXammy.com וכותבת סדרת מאמרי ממשק ב-Telerik). תיעוד רשמי ומקורות נוספים להעמקה:

- Microsoft Learn: [מדריך הפיתוח הרשמי ל-.NET MAUI](#)
- Microsoft Learn: [עקרונות Data Binding וארכיטקטורת MVVM](#)
- Microsoft Learn: [ממשק ICommand ותנאי CanExecute](#)
- Microsoft Learn: [מצבי קישור נתונים \(Binding Modes\) ב-.NET MAUI](#)

פרק הרחבה – מדריך פיתוח מפורט: -PlayListSample StepByStep (שיטת הבלוקים המודולרית)

מדריך פדגוגי מודולרי לבניית אפליקציית .NET MAUI שלמה צעד-אחר-צעד, בהשראת מתודולוגיית הבלוקים (The Block Strategy)

🎯 חלק ראשון: מה מטרת המדריך?

1.1 למי נועד המדריך ומה נבנה בו?

ברוכים הבאים לעולם פיתוח האפליקציות ב-.NET MAUI! אם הגעתם לכאן, כנראה שכבר שמעתם את המושג **MVVM** (Model-View-ViewModel – דפוס הארכיטקטורה המוביל בתעשייה לפיתוח ממשקי משתמש מודרניים. עבור מפתחים ותלמידים בתחילת הדרך, המפגש הראשון עם ארכיטקטורת שכבות עלול להיראות מאיים: פתאום לא כותבים קוד בתוך כפתור, יש קבצים שונים, מושגים חדשים כמו `BindingContext`, `INotifyPropertyChanged`, `ICommand` ו-`Service`.

מטרת מדריך זה היא להוריד את מפלס החרדה ולבנות ביטחון מוחלט בפיתוח. במקום להעמיס 300 שורות קוד בבת אחת, נפרק את בניית האפליקציה **PlayListSample** למנות קטנות, הגיוניות וקלות לעיכול:

- נתחיל מהצורך הוויזואלי ומהסקיצה של המסך.
- נבנה תחילה **אך ורק את מנגנון הדפדוף בסרטונים** (החלק העליון של המסך) עם נתונים מקומיים בסיסיים, ללא שירותים מורכבים וללא טופס.

- נפרק הן את ה-ViewModel והן את ה-XAML לבלוקים קטנים וממוקדים, המלווים בטבלאות הסבר מפורטות לכל שדה, פקודה וחיבור.
- נוודא שהכול עובד, מתחבר ומובן.
- רק לאחר מכן נשדרג את המערכת לשכבת שירות מקצועית (Service), ולבסוף נוסיף את טופס הזנת סרטון חדש בפירוק מודולרי דומה.

בסיום המדריך תהיה בידיכם אפליקציית פלייליסט סרטונים עובדת, יפהפייה, בנויה לפי הסטנדרטים הגבוהים ביותר של פיתוח מובייל.

שאלות הבנה ובדיקה עצמית – חלק ראשון

נסו להשיב בעצמכם לפני שתמשיכו לשלב הבא:



1. מדוע באפליקציות מובייל מקצועיות מפרידים בין הלוגיקה (ViewModel) לבין עיצוב המסך (View), במקום לכתוב את כל הקוד בקובץ הקוד שמאחורי המסך (Code-Behind)? (רמז: חשבו מה קורה כשרוצים להחליף עיצוב, או כשרוצים לבדוק את הלוגיקה בלי להפעיל אמולטור).
2. מהו החיסרון המרכזי של כתיבת לוגיקה עסקית באירוע `Button_Clicked` בהשוואה לשימוש בפקודות (`Command`)?

חלק שני: מתודולוגיית הפיתוח של המדריך

2.1 "שביל הזהב": שרשרת הפיתוח הנכונה

אחת השאלות הנפוצות ביותר אצל מתחילים היא: "יש לי רעיון למסך – מאיפה מתחילים לכתוב קוד?" האם מתחילים ישר בעיצוב ה-XAML? האם קודם כותבים את מחלקות ה-C#?

הניסיון המקצועי מלמד שהדרך הבטוחה והנקייה ביותר להימנע מבאגים ומבלבול היא מתודולוגיית "שביל הזהב" המודולרית:

TEXT

1. ב-2 דקות (Wireframe) שרטוט סקיצה מהירה על דף	
2. מה הישות שלנו – (Model) הגדרת מודל הנתונים	
3. פקודות ורענון, State-פירוק ל – ViewModel-מחשבת ה	
4. (View) XAML-והלבשת המסך ב BindingContext-חיבור ה	
5. בדיקה ראשונית והרצה של מנגנון הדפדוף	
6. שדרוג לשכבת שירות (Service & Mockup)	
7. (View-וב VM-טופס הוספה חדש ב) הרחבת היכולות	

2.2 עקרונות הפירוק המודולרי לבלוקים (The Modular Block Strategy)

לאורך כל המדריך לא נציג אף קובץ כגוש קוד ארוך, אלא נפעל לפי עקרון הבלוקים:

1. **פירוק ה-ViewModel לפונקציונליים:** שדות פרטיים → מאפיינים פומביים → פקודות → פעולה בונה → מתודות ביצוע ו-CanExecute → מתודת עדכון ורענון מרכזי (`RefreshCommands`).
2. **טבלאות ייעודיות לכל שכבה:** טבלת תפקיד שדות המודל, טבלת פקודות ותנאי CanExecute, וטבלת מיפוי בין ה-XAML ל-ViewModel.
3. **פירוק ה-XAML לתת-בלוקים:** הגדרת ראש דף ו-Compiled Bindings → שלד הפריסה → כרטיסיית ה-Border → כותרת → סרגל דפדוף בגריד → קו מפריד → פרטי סרטון → כפתורי פעולה.
4. **שבירת שורות תקנית (Attribute-per-line indent):** בכל תגית המכילה 2 תכונות ומעלה – יורדים שורה עבור כל תכונה כדי למנוע גלישת טקסט ב-PDF ובמסך.

שאלות הבנה ובדיקה עצמית – חלק שני

נסו להשיב בעצמכם לפני שתמשיכו לשלב הבא:

1. כיצד שרטוט מהיר של המסך על דף נייר מסייע לנו לדעת בדיוק אילו מאפיינים (`Properties`) ואילו פקודות (`Commands`) נצטרך לכתוב ב-ViewModel?
2. מה עלול לקרות אם נתחיל מיד בעיצוב ה-XAML לפני שהגדרנו את שמות המאפיינים והפקודות ב-ViewModel?

חלק שלישי: הגדרת האפליקציה ובנייתה בשלבים

סקיצת המסך (Screen Wireframe) ומפת הבלוקים

לפני שניגש לכתיבת שורת קוד אחת, נבחן את סקיצת המסך של אפליקציית **PlaylistSample**. המסך מחולק לשני בלוקים עיקריים:



09:41

ניהול פלייליסט - חזרה על MVVM

דפדוף, פקודות CanExecute וטופס הזנה Two-Way

בלוק 1: דפדוף בסרטונים (ITEM VIEWER)

דפדוף בסרטונים (תצוגת פריט בודד) 

→ הבא

סרטון 2 מתוך 4

← הקודם

מבוא ל-Data Binding ב-MAUI

כותרת:

DotNetIL

ערוץ:

24 דק'

משך:

youtu.be/sample-binding

כתובת:

נצפה ✓

סטטוס:

שנה סטטוס צפייה 

בלוק 2: טופס הזנה (TWO-WAY BINDING)

+ הוספת סרטון חדש (טופס קלט)

כותרת הסרטון:

פקודות ICommand ו-CanExecute

שם הערוץ:

Microsoft Developers

משך בדקות:

18

כתובת אינטרנט (URL):

https://youtube.com/watch?v=demo2

נקה טופס 

שמור סרטון 

הסרטון 'פקודות ICommand' נוסף בהצלחה לרשימה!

ניתוח מבנה הסקיצה:

- **בלוק 1 (המסגרת הכחולה העליונה):** כרטיסיית דפדוף וצפייה בסרטון בודד. מכילה סרגל ניווט ([הקודם](#) < , מונה מיקום, [הבא](#) >), פרטי הסרטון הנוכחי, וכפתור לשינוי סטטוס צפייה.
- **בלוק 2 (המסגרת הירוקה התחתונה):** כרטיסיית טופס הזנה. מכילה שדות קלט עבור כותרת, ערוץ, משך זמן וכתובת URL, לצד כפתורי שמירה ואיפוס.

החלטה פדגוגית מכרעת: אנו **לא** בונים כרגע את בלוק 2 (הטופס התחתון)! כל שלב א' יוקדש כולו, מקצה לקצה, **אך ורק לבלוק 1 (דפדוף בסרטונים)**. נלמד להציג את הסרטון הנוכחי ולדפדף קדימה ואחורה בצורה יציבה ומוגנת.

 **שלב א': מנגנון דפדוף בסרטונים (תצוגה מקומית ללא Service)**

צעד 1: הגדרת מודל הנתונים (`VideoItem.cs`)

מודל (Model) הוא מחלקת C# פשוטה (POCO – Plain Old CLR Object) שתפקידה לייצג את הנתונים העסקיים. המודל אינו מכיר את המסך ואינו יודע כיצד מציגים אותו.

טבלת ניתוח שדות המודל:

לפני שנכתוב את הקוד, נבין מה תפקידו של כל שדה במחלקה:

שדה / מאפיין	טיפוס (Type)	תפקיד ומשמעות עסקית	דוגמה לערך
Id	int	מזהה ייחודי עבור הסרטון (מפתח ראשי עתידי במסד נתונים)	1, 2
Title	string	כותרת הסרטון	"NET MAUI-מבוא מעשי ל"
ChannelName	string	שם הערוץ או היוצר של הסרטון	"Tech Academy"
DurationMinutes	int	משך הזמן בדקות (ערך מספרי לצורך חישובים)	12
Url	string	כתובת אינטרנט מלאה לצפייה בסרטון	"https://youtu.be/maui-intro"
IsWatched	bool	האם הסרטון נצפה כבר על ידי המשתמש (false / true)	false
FormattedDuration	string (קריאה בלבד)	מאפיין עזר המעצב את הדקות למחרוזת מוכנה לתצוגה	"דקות 12"
WatchedStatusText	string (קריאה בלבד)	מאפיין עזר הממיר בוליאני למחרוזת עברית ידידותית	"נצפה" / "טרם נצפה"

ניצור בתיקית **Models** את הקובץ **VideoItem.cs**:

בלוק 1.1: מזהה ושדות המידע הבסיסיים

```

CSHARP

namespace PlaylistSample.Models;

public class VideoItem
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public string Url { get; set; } = string.Empty;
    public string ChannelName { get; set; } = string.Empty;
    public int DurationMinutes { get; set; }
    public bool IsWatched { get; set; }
}

```

בלוק 1.2: מאפייני עזר לתצוגה נקייה

```

CSHARP

// -מאפייני עזר לקריאה בלבד - מכינים את המידע לתצוגה נקייה ב
public string FormattedDuration => $"{DurationMinutes} דקות";

```

```
public string WatchedStatusText => IsWatched ? "טרם נצפה" : "נצפה";
}
```

פינת העמקה: מאפייני עזר במודל במקום ממירי ערכים

שימו לב לשני המאפיינים `FormattedDuration` ו-`WatchedStatusText`. במקום לכתוב ב-XAML ממיר ערכים מורכב (`ValueConverter`), המודל עצמו מספק את הייצוג הטקסטואלי הנוח להצגה. זוהי דרך פשוטה, קריאה ויזוית ביותר למתחילים!

שאלות הבנה לשלב 1:

- מדוע מחלקת המודל אינה מכילה שום פקדי ממשק (כמו `Label` או `Button`)?
- מה היתרון בהגדרת מאפיין עזר כגון `FormattedDuration` בתוך המודל?

צעד 2: הגדרת ה-ViewModel בשיטת הבלוקים (שלד ראשוני ללא Service)

ה-ViewModel מתווך בין המודל לבין המסך: הוא מחזיק את הנתונים שהמסך צריך להציג בכל רגע, ומנהל את הפעולות שהמשתמש יכול לבצע.

1. מחלקת הבסיס (`BaseViewModel.cs`)

כדי שהמסך יתעדכן כשאנו משנים ערך ב-C#, המחלקה חייבת לממש את ממשק `INotifyPropertyChanged`. ניצור בתיקיית `ViewModels` את `BaseViewModel.cs`:

CSHARP

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace PlaylistSample.ViewModels;

public abstract class BaseViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;

    private string _title = string.Empty;
    public string Title
    {
        get => _title;
        set
        {
            if (_title != value)
            {
                _title = value;
                OnPropertyChanged();
            }
        }
    }
}
```

```

private bool _isLoading;
public bool IsLoading
{
    get => _isLoading;
    set
    {
        if (_isLoading != value)
        {
            _isLoading = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(IsNotLoading));
        }
    }
}

public bool IsNotLoading => !IsLoading;

protected void OnPropertyChanged([CallerMemberName] string? propertyName = null)
{
    PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
}
}

```

2. טבלת ניתוח מאפייני המצב (State Properties) של שלב א':

מאפיין ב-ViewModel	טיפוס	תפקיד בממשק המשתמש (UI)	מתי המאפיין מתעדכן?
CurrentVideo	VideoItem?	מחזיק את כל אובייקט הסרטון שמוצג כעת במסך	בעת דפדוף, הוספה או טעינה
CurrentPositionText	string	מציג את מונה המיקום הנוכחי ("סרטון 1 מתוך 3")	בכל שינוי אינדקס בסרגל הניווט
Title	string	כותרת המסך העליונה	באתחול ה-ViewModel
IsLoading	bool	שולט במחווין הטעינה (ActivityIndicator)	בפעולות א-סינכרוניות

3. טבלת ניתוח הפקודות (Commands) של שלב א':

פקודה (IComm and)	פקד מפעיל ב-XAML	פעולת ביצוע (Execute)	תנאי ביצוע (CanExecute)	מה קורה כשהתנאי שקרי (false)?
NextCommand	כפתור → הבא	ExecuteNext(): קידום האינדקס ב-1 וטעינת הסרטון הבא	_currentIndex < _videos.Count - 1 && IsNotLoading	הכפתור הופך לאפור/מנוטרל בסוף הרשימה
PreviousCommand	כפתור ← הקודם	ExecutePrevious(): החזרת האינדקס ב-1 וטעינת הסרטון הקודם	_currentIndex > 0 && IsNotLoading	הכפתור הופך לאפור/מנוטרל בסרטון הראשון
ToggleWatchedCommand	כפתור שנה סטטוס צפייה	ExecuteToggleWatched(): הפיכת ערך IsWatched ורענון התצוגה	CurrentVideo != null && IsNotLoading	הכפתור מנוטרל אם אין סרטון פעיל

4. פירוק ה-MainViewModel ל-6 בלוקים לוגיים מודולריים

במקום לכתוב את המחלקה ברצף ארוך, נרכיב אותה בלוק אחר בלוק:

אלו המשתנים הפנימיים שה-ViewModel שומר בזיכרון:

CSHARP

```
using System.Windows.Input;
using PlaylistSample.Models;

namespace PlaylistSample.ViewModels;

public class MainViewModel : BaseViewModel
{
    // רשימת הסרטונים המקומית (זמנית בבגאי לצורך שלב א')
    private readonly List<VideoItem> _videos;
    private int _currentIndex = 0;

    // שדות המצב של התצוגה
    private VideoItem? _currentVideo;
    private string _currentPositionText = "טנען נתונים";
}
```

מאפיינים אלו נחשפים ל-XAML. שימו לב שהם כוללים setter המפעיל את `OnPropertyChanged()`

CSHARP

```
public VideoItem? CurrentVideo
{
    get => _currentVideo;
    private set
    {
        if (_currentVideo != value)
        {
            _currentVideo = value;
            OnPropertyChanged();
        }
    }
}

public string CurrentPositionText
{
    get => _currentPositionText;
    private set
    {
        if (_currentPositionText != value)
        {
            _currentPositionText = value;
            OnPropertyChanged();
        }
    }
}
```

פינת העמקה: למה בחלק הראשון יש `CurrentVideo` ובחלק השני שדות נפרדים?



בחלק הדפדוף (בלוק 1), אנו מציגים ישות קיימת מתוך הרשימה. לכן, טבעי ונקי להחזיק אובייקט שלם מסוג `CurrentVideo`, ומתוכו ה-XAML שולף `{Binding CurrentVideo.Title}` וכו'. לעומת זאת, בטופס ההוספה (בלוק 2 שנבנה בשלב ג'), המשתמש מקליד נתונים חדשים שטרם נוצר עבורם אובייקט! שם נדרשים שדות קלט בדידים (`InputTitle`, `InputChannelName`) שיתחברו בקישור דו-כיווני (`Two-Way`).

בלוק 3: הצהרה על הפקודות (Commands Declaration)

אנו מגדירים פקודות מסוג `ICommand` שאליהן יתחברו הכפתורים במסך:

CSHARP

```
// פקודות ניווט וסטטוס
public ICommand NextCommand { get; }
public ICommand PreviousCommand { get; }
public ICommand ToggleWatchedCommand { get; }
```

בלוק 4: הפעולה הבונה (Constructor)

כאן מאתחלים את הנתונים המקומיים, מקשרים את הפקודות למתודות הביצוע ולתנאי `CanExecute`, ומעדכנים את התצוגה הראשונית:

CSHARP

```
public MainViewModel()
{
    Title = "MVVM פלייליסט סרטונים - חזרה על";

    // אתחול נתונים מקומיים ישירות בבנאי 1.
    _videos = new List<VideoItem>
    {
        new VideoItem
        {
            Id = 1,
            Title = "NET MAUI-מבוא מעשי ל",
            ChannelName = "Tech Academy",
            DurationMinutes = 12,
            Url = "https://youtu.be/maui-intro",
            IsWatched = false
        },
        new VideoItem
        {
            Id = 2,
            Title = "MVVM הבנת ארכיטקטורת",
            ChannelName = "Code Masters",
            DurationMinutes = 18,
        }
    }
}
```

```

        Url = "https://youtu.be/mvvm-explained",
        IsWatched = true
    },
    new VideoItem
    {
        Id = 3,
        Title = "חיבור נתונים ופקודות (Data Binding)",
        ChannelName = "Dev Israel",
        DurationMinutes = 15,
        Url = "https://youtu.be/binding-deep-dive",
        IsWatched = false
    }
};

_currentIndex = 0;

// 2. קישור הפקודות למתודות הביצוע ולתנאי CanExecute
NextCommand = new Command(ExecuteNext, CanExecuteNext);
PreviousCommand = new Command(ExecutePrevious, CanExecutePrevious);
ToggleWatchedCommand = new Command(ExecuteToggleWatched, () => CurrentVideo != null &&
IsNotLoading);

// 3. הצגת הסרטון הראשון במסך
UpdateCurrentVideo();
}

```

 **בלוק 5: מתודות ביצוע הפקודות ותנאי ה-CanExecute**

לכל פקודה יש מתודת ביצוע (**Execute**) ומתודת תנאי (**CanExecute**):

CSHARP

```

private void ExecuteNext()
{
    if (_currentIndex < _videos.Count - 1)
    {
        _currentIndex++;
        UpdateCurrentVideo();
    }
}

private bool CanExecuteNext()
{
    return IsNotLoading && _videos.Count > 0 && _currentIndex < _videos.Count - 1;
}

private void ExecutePrevious()
{
    if (_currentIndex > 0)
    {
        _currentIndex--;
        UpdateCurrentVideo();
    }
}

```

```

private bool CanExecutePrevious()
{
    return IsNotLoading && _videos.Count > 0 && _currentIndex > 0;
}

private void ExecuteToggleWatched()
{
    if (CurrentVideo == null) return;

    CurrentVideo.IsWatched = !CurrentVideo.IsWatched;
    OnPropertyChanged(nameof(CurrentVideo));
}

```

🧩 בלוק 6: מתודת עדכון התצוגה ורענון הפקודות המרכזי

מתודות אלו מעדכנות את המאפיינים ואת מצב הכפתורים במסך:

CSHARP

```

private void UpdateCurrentVideo()
{
    if (_videos.Count > 0 && _currentIndex ≥ 0 && _currentIndex < _videos.Count)
    {
        CurrentVideo = _videos[_currentIndex];
        CurrentPositionText = $"סרטון {_currentIndex + 1} מתוך {_videos.Count}";
    }
    else
    {
        CurrentVideo = null;
        CurrentPositionText = "אין סרטונים זמינים";
    }

    // רענון מרכזי של מצב הכפתורים במסך
    RefreshCommands();
}

/// <summary>
/// שלחן CanExecute-מאותת לפקודות לחשב מחדש את תנאי ה
/// </summary>
private void RefreshCommands()
{
    ((Command)NextCommand).ChangeCanExecute();
    ((Command)PreviousCommand).ChangeCanExecute();
    ((Command)ToggleWatchedCommand).ChangeCanExecute();
}
}

```



פינת העמקה: מהי הפעולה `ChangeCanExecute()` ולמה ריכזנו אותה ב- `RefreshCommands()` ?

לכל פקודת `Command` ב-MAUI יש תנאי הפעלה (`CanExecute`). אך מאיפה המסך יודע מתי לבדוק מחדש את התנאי הזה? בדיוק לשם כך נועדה הפעולה `ChangeCanExecute()` ! היא מפעילה את האירוע `CanExecuteChanged`, ומאותתת לפקדים ב-UI (כגון כפתורים) לחשב מחדש האם עליהם להיות זמינים או מנוטרלים (אפורים). ריכוז כל הקריאות הללו בתוך מתודה יחידה `RefreshCommands()` הוא דפוס פיתוח מקצועי: בכל שינוי אינדקס או מצב טעינה, אנו קוראים למתודה זו ומבטיחים שכל כפתורי המסך יסתנכרו מיד ובבת אחת!

שאלות הבנה לשלב 2:

- איזו שגיאת זמן ריצה נמנעת הודות לשימוש בתנאי `CanExecuteNext` כאשר המשתמש מגיע לסוף הרשימה ומנסה ללחוץ שוב ושוב על "הבא"?
- מדוע ריכוז הקריאות ל- `ChangeCanExecute` תחת פעולה מרכזית `RefreshCommands()` עדיף על פני פיזורן במקומות שונים בקוד?

צעד 3: ה-View וחיבור הנתונים (שיטת הבלוקים המודולרית ב-XAML)

כעת נבנה את מסך ה-XAML. לפי מתודולוגיית הבלוקים (*The Block Strategy*), לא נזרוק קובץ שלם בבת אחת! נחבר תחילה את קובץ הקוד שמאחורי המסך, ולאחר מכן נרכיב את ה-XAML בתת-בלוקים מובנים.

שלב 3.1: חיבור ה- `BindingContext` בקוד האחורי (`Views/MainPage.xaml.cs`)

לפני שנכתוב ביטויי `{Binding}` ב-XAML, המסך חייב לדעת מי מספק לו את הנתונים. נפתח את `MainPage.xaml.cs` ונגדיר את ה-`ViewModel` כהקשר הנתונים של הדף:

CSHARP

```
using PlaylistSample.ViewModels;

namespace PlaylistSample.Views;

public partial class MainPage : ContentPage
{
    public MainPage()
    {
        InitializeComponent();

        // כהקשר הנתונים ViewModel-יצירה והקצאה ישירה של ה
        BindingContext = new MainViewModel();
    }
}
```

פינת העמקה: מהו BindingContext?

המאפיין BindingContext הוא "מקור האמת" של הדף. ברגע שקבענו אותו כ- `new MainViewModel()`, כל פקד ב-XAML שמכיל `{Binding SomeProperty}` יודע אוטומטית לחפש את המאפיין הזה בתוך מופע ה- `MainViewModel`!

שלב 3.2: הגדרת ראש הדף, מרחבי שמות ו-Compiled Bindings

נפתח את `MainPage.xaml`. שימו לב במיוחד להבדל בין השורות שנוצרות אוטומטית לבין השורות שחובה עליכם להוסיף בעצמכם:

XML

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
  xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"

  <!-- (1) מרחב השמות ל: (2) חובה: 1: ViewModels-תוספת חובה: 1: מרחב השמות ל: -->
  xmlns:vm="clr-namespace:PlayListSample.ViewModels"

  x:Class="PlayListSample.Views.MainPage"

  <!-- (2) חובה: 2: חיבור: Compiled Bindings-תוספת חובה: 2: חיבור: -->
  x:DataType="vm:MainViewModel"

  <!-- (3) חובה: 3: כיוון מימין לשמאל וכותרת מקושרת: -->
  Title="{Binding Title}"
  FlowDirection="RightToLeft">

  <!-- כאן יבוא שלד הפריסה הראשי -->

</ContentPage>
```

מה התלמיד חייב להוסיף בעצמו בהגדרת הדף?

1. שורה זו אינה נוצרת מעצמה! מילת המפתח `clr-namespace:` מייבאת את תיקיית ה-`ViewModels` של הפרויקט לתוך ה-XAML תחת הכינוי `vm`.
2. שורה קריטית המפעילה את מנגנון ה-`Compiled Bindings`! ללא שורה זו, כל שגיאת איות במאפיין (כמו `{Binding CurentVideo}`) תתגלה רק בריצה כשהטקסט לא יופיע. עם שורה זו, הקומפילר בודק מראש שכל שדה קיים, מונע שגיאות ומעניק השלמה אוטומטית (IntelliSense).
3. `FlowDirection="RightToLeft"`: מבטיח פריסה עברית נכונה מימין לשמאל.

נוסיף בתוך הדף `ScrollView` המאפשר גלילה במכשירים קטנים, עם עוגני מקום עבור הבלוקים:

XML

```
<ScrollView>
  <VerticalStackLayout Padding="20"
    Spacing="16">

    <!-- כותרת ראשית ומחווין טעינה -->
    <Label Text="{Binding Title}"
      FontSize="22"
      FontAttributes="Bold"
      HorizontalOptions="Center" />

    <ActivityIndicator IsRunning="{Binding IsLoading}"
      IsVisible="{Binding IsLoading}"
      HorizontalOptions="Center" />

    <!-- בלוק 1: כרטיסיית דפדוף בסרטונים (נרכיב כעת בתת-בלוקים) -->
    <!-- בלוק 2: כרטיסיית טופס הזנה (נרכיב בשלב ג') -->

  </VerticalStackLayout>
</ScrollView>
```

שלב 3.4: הרכבת בלוק 1 בתת-בלוקים מודולריים

במקום לכתוב את כל ה- `Border` כיחידה אחת, נרכיב אותו תת-בלוק אחר תת-בלוק:

דפדוף בסרטונים (תצוגת פריט בודד)

→ הבא

סרטון 2 מתוך 4

← הקודם

מבוא ל-Data Binding ב-MAUI

כותרת:

DotNetIL

ערוץ:

24 דק'

משך:

youtu.be/sample-binding

כתובת:

נצפה ✓

סטטוס:

שנה סטטוס צפייה 

♦ תת-בלוק 1.1: מעטפת הכרטיסייה והגדרת הפינות המעוגלות

אנו משתמשים בפקד `Border` עם פינות מעוגלות ברידיוס 8:

XML

```
<Border Padding="14"
  StrokeShape="RoundRectangle 8">
  <VerticalStackLayout Spacing="10">
    <!-- כאן יבואו מרכיבי הכרטיסייה הפנימיים -->
  </VerticalStackLayout>
</Border>
```

♦ תת-בלוק 1.2: כותרת הכרטיסייה

תווית פנימית המגדירה למשתמש את מהות הכרטיסייה:

XML

```
<Label Text="(תצוגת פריט בודד)"
        FontSize="16"
        FontAttributes="Bold" />
```

♦ תת-בלוק 1.3: סרגל הדפדוף העליון (גריד 3 עמודות)

גריד המחלק את הרוחב לשלושה אזורים: כפתור ימני, מונה מרכזי וכפתור שמאלי:

XML

```
<!-- סרגל דפדוף עליון -->
<Grid ColumnDefinitions="Auto,*,Auto">
    <Button Grid.Column="0"
            Text="← הקודם"
            Command="{Binding PreviousCommand}" />

    <Label Grid.Column="1"
           Text="{Binding CurrentPositionText}"
           FontSize="15"
           FontAttributes="Bold"
           HorizontalOptions="Center"
           VerticalOptions="Center" />

    <Button Grid.Column="2"
            Text="→ הבא"
            Command="{Binding NextCommand}" />
</Grid>
```

♦ תת-בלוק 1.4: קו מפריד עדין

קו גרפי דק בגובה 1 פיקסל להפרדה אסתטית:

XML

```
<BoxView HeightRequest="1"
        Color="LightGray" />
```

♦ תת-בלוק 1.5: פרטי הסרטון הנוכחי מתוך `CurrentVideo` עם עיצוב מחרוזות*

תוויות המציגות את כותרת הסרטון, הערוץ, משך הזמן והכתובת:

XML

```

<!-- פרטי הפריט הנוכחי המוצג על המסך מתוך CurrentVideo -->
<Label Text="{Binding CurrentVideo.Title, StringFormat='{0} : כותרת:}'"
        FontSize="15"
        FontAttributes="Bold" />

<Label Text="{Binding CurrentVideo.ChannelName, StringFormat='{0} : ערוץ:}'" />

<Label Text="{Binding CurrentVideo.FormattedDuration, StringFormat='{0} : משך:}'" />

<Label Text="{Binding CurrentVideo.Url, StringFormat='{0} : כתובת אינטרנט:}'"
        TextColor="DarkBlue"
        LineBreakMode="TailTruncation" />

<Label Text="{Binding CurrentVideo.WatchedStatusText, StringFormat='{0} : סטטוס:}'" />

```

* הרחבה על `StringFormat` ב-`Data Binding`: המאפיין `StringFormat='{0} : כותרת:'` מאפשר לשלב טקסט קבוע מראש לצד הנתון הדינמי המגיע מה-`ViewModel`, כאשר `{0}` מייצג את המקום שבו יישתל הערך. הדבר חוסך שרשורי מחרוזות ידניים בקוד C#. [לתיעוד המלא של מיקרוסופט: Microsoft Learn - String formatting in .NET MAUI Data Binding](#)

♦ תת-בלוק 1.6: כפתור שינוי סטטוס צפייה

כפתור המקושר ישירות לפקודת ה-`ToggleWatchedCommand`:

XML

```

<Button Text="שנה סטטוס צפייה"
        Command="{Binding ToggleWatchedCommand}" />

```

תפקיד בממשק	ביטוי Data Binding	תכונה מקושרת	פקד ב-XAML
מעבר לסרטון הקודם	<code>{Binding PreviousCommand}</code>	Command	Button (הקודם)
הצגת מיקום הסרטון ("1 מתוך 3")	<code>{Binding CurrentPositionText}</code>	Text	Label (מונה)
מעבר לסרטון הבא	<code>{Binding NextCommand}</code>	Command	Button (הבא)
הצגת כותרת הסרטון	<code>{Binding CurrentVideo.Title, StringFormat='{0} כותרת:'}</code>	Text	Label (כותרת)
הצגת שם הערוץ	<code>{Binding CurrentVideo.ChannelName, StringFormat='{0} ערוץ:'}</code>	Text	Label (ערוץ)
הצגת משך הזמן המעוצב	<code>{Binding CurrentVideo.FormattedDuration, StringFormat='{0} משך:'}</code>	Text	Label (משך)
הצגת קישור ה-URL	<code>{Binding CurrentVideo.Url, StringFormat='{0} כתובת אינטרנט:'}</code>	Text	Label (כתובת)
הצגת סטטוס הצפייה	<code>{Binding CurrentVideo.WatchedStatusText, StringFormat='{0} סטטוס:'}</code>	Text	Label (סטטוס)
החלפת מצב נצפה/טרם נצפה	<code>{Binding ToggleWatchedCommand}</code>	Command	Button (סטטוס)

שאלות הבנה לשלב 3:

- מה היה קורה אילו שכחנו להגדיר `BindingContext` בקובץ `MainPage.xaml.cs`?
- מה היתרון בשימוש ב- `StringFormat='{0} כותרת:'` ישירות ב-XAML במקום לבצע שרשורי טקסט ב-`StringFormat='{0} כותרת:'`?

צעד 4: בדיקות והרצה ראשונית של שלב א'

הגענו לרגע הבדיקה של שלב א'! נריץ את האפליקציה:

- הפעלה:** המסך נפתח, מציג את "סרטון 1 מתוך 3", ואת פרטי הסרטון הראשון.
- בדיקת גבול התחלה:** כפתור "← הקודם" מנוטרל ואפור! לחיצה עליו אינה מבצעת דבר כי `CanExecutePrevious` החזיר `false`.
- דפדוף קדימה:** לחיצה על "הבא →" מעדכנת את `CurrentVideo` לסרטון השני, מונה המיקום הופך ל-"סרטון 2 מתוך 3", וכפתור "← הקודם" הופך לפעיל מיד!
- בדיקת גבול סוף:** בלחיצה נוספת מגיעים לסרטון השלישי. כעת כפתור "הבא →" מנוטרל אוטומטית.
- סטטוס צפייה:** לחיצה על "שנה סטטוס צפייה" מחליפה מיידית בין "נצפה" ל"טרם נצפה".

שאלות הבנה לשלב 4:

- כיצד מנגנון ה- `CanExecute` מגן על האפליקציה מקריסת `ArgumentOutOfRangeException`?
- מדוע חשוב לבדוק את המסך ביסודיות בשלב זה לפני שניגשים לשכתב השירות ולטופס?

מדוע עוברים לשירות (Service) ומדוע משתמשים ב-Mockup?

בשלב א' החזקנו את רשימת הסרטונים בזיכרון של ה-ViewModel. זה עבד נהדר להבנה ראשונית, אך באפליקציות אמיתיות נתונים נשמרים במסד נתונים (כמו SQLite) או בשרת ענן.

מדוע איננו מחברים ישר מסד נתונים או שרת אינטרנט?

1. **בידוד משתנים פדגוגי (Isolation of Concerns):** כשתלמיד לומד ארכיטקטורת MVVM, עירוב של התקנת חבילות NuGet של SQLite, כתיבת שאילתות SQL או טיפול בבעיות תקשורת רשת יגרום לתסכול ויקשה לדעת מאיזו שכבה נובע באג.
 2. **פיתוח מונחה ממשק (Interface-Driven Development):** אנו מגדירים ממשק `IVideoService`. ה-ViewModel יתקשר אך ורק מול הממשק.
 3. **החלפה חלקה בעתיד:** מחר, כשנעבור ל-SQLite, נכתוב מחלקה חדשה `SqliteVideoService` הממשת את אותו `IVideoService`. שורת הקוד היחידה שתשתנה תהיה באתחול המסך – בעוד שכל ה-ViewModel וה-XAML יישארו ללא שינוי של תו בודד!
-

צעד 1: הגדרת ממשק השירות (`IVideoService.cs`)

טבלת ניתוח פעולות הממשק:

מטודה וחתימה בממשק	טיפוס מוחזר	פרמטרים	תפקיד ומשמעות עסקית	מדוע Task א-סינכרוני?
<code>GetAllVideosAsync()</code>	<code>Task<IReadOnlyList<VideoItem>></code>	ללא	שליפת כל הסרטונים הקיימים במאגר	הדמיית זמן גישה לדיסק או לרשת
<code>AddVideoAsync(video)</code>	<code>Task<bool></code>	<code>VideoItem</code> <code>video</code>	הוספת סרטון חדש והקצאת מזהה (Id)	הדמיית שמירה בקובץ או במסד נתונים
<code>UpdateVideoAsync(video)</code>	<code>Task<bool></code>	<code>VideoItem</code> <code>video</code>	עדכון סרטון קיים (למשל סטטוס צפייה)	הדמיית עדכון רשומה קיימת במאגר

ניצור בתיקית `Services` את הממשק `IVideoService.cs`:

CSHARP

```
using PlaylistSample.Models;

namespace PlaylistSample.Services;

public interface IVideoService
{
    // שליפת כל הסרטונים באופן א-סינכרוני
    Task<IReadOnlyList<VideoItem>> GetAllVideosAsync();

    // הוספת סרטון חדש באופן א-סינכרוני
    Task<bool> AddVideoAsync(VideoItem video);

    // עדכון סרטון קיים (למשל שינוי סטטוס צפייה)
    Task<bool> UpdateVideoAsync(VideoItem video);
}
```

פינת העמקה: מדוע כל הפעולות מחזירות `Task` א-סינכרוני?

פעולות קריאה וכתובה לדיסק או לרשת דורשות זמן. אם נבצע אותן באופן סינכרוני, האפליקציה "תקפא" והמשתמש לא יוכל ללחוץ על כלום! שימוש ב- `Task` ו- `async/await` מאפשר לבצע את הפעולה ברקע בזמן שהמסך נשאר רספונסיבי לחלוטין.

ניצור בתיקיית **Services** שירות מדומה (Mockup) המחזיק רשימה סטטית ומדמה השהיית רשת קצרה בעזרת **Task.Delay**:

CSHARP

```
using PlaylistSample.Models;

namespace PlaylistSample.Services;

public class MockVideoService : IVideoService
{
    private static readonly List<VideoItem> _storage = new()
    {
        new VideoItem
        {
            Id = 1,
            Title = "NET MAUI-מבוא מעשי ל",
            ChannelName = "Tech Academy",
            DurationMinutes = 12,
            Url = "https://youtu.be/maui-intro",
            IsWatched = false
        },
        new VideoItem
        {
            Id = 2,
            Title = "MVVM הבנת ארכיטקטורת",
            ChannelName = "Code Masters",
            DurationMinutes = 18,
            Url = "https://youtu.be/mvvm-explained",
            IsWatched = true
        },
        new VideoItem
        {
            Id = 3,
            Title = "(Data Binding) חיבור נתונים ופקודות",
            ChannelName = "Dev Israel",
            DurationMinutes = 15,
            Url = "https://youtu.be/binding-deep-dive",
            IsWatched = false
        }
    };

    public async Task<IReadOnlyList<VideoItem>> GetAllVideosAsync()
    {
        // סימולציית המתנה קצרה כמו גישה לדיסק
        await Task.Delay(150);
        return _storage.ToList().AsReadOnly();
    }

    public async Task<bool> AddVideoAsync(VideoItem video)
    {
        await Task.Delay(150);
        video.Id = _storage.Count > 0 ? _storage.Max(v => v.Id) + 1 : 1;
        _storage.Add(video);
        return true;
    }
}
```

```

}

public async Task<bool> UpdateVideoAsync(VideoItem video)
{
    await Task.Delay(100);
    var existing = _storage.FirstOrDefault(v => v.Id == video.Id);
    if (existing != null)
    {
        existing.Title = video.Title;
        existing.ChannelName = video.ChannelName;
        existing.DurationMinutes = video.DurationMinutes;
        existing.Url = video.Url;
        existing.IsWatched = video.IsWatched;
        return true;
    }
    return false;
}
}

```

צעד 3: שכתוב ה-ViewModel לקבלת השירות וטעינה א-סינכרונית נקייה

ב-C#, פעולה בונה (Constructor) אינה יכולה להיות `async`. לכן, כיצד נטען נתונים א-סינכרונית בעת יצירת ה-ViewModel מבלי להשתמש בתחביר מבלבל? נשמור את משימת הטעינה בשדה `private Task load;` ונקרא למתודה מסודרת `LoadVideosAsync()` המנהלת את מצב הטעינה ומטפלת בשגיאות:

CSHARP

```

// בתוך MainViewModel.cs:
private readonly IVideoService _videoService;
private readonly List<VideoItem> _videos = new();
private Task load;

public MainViewModel(IVideoService videoService)
{
    _videoService = videoService;
    Title = "MVVM ניהול פלייליסט - חזרה על";

    NextCommand = new Command(ExecuteNext, CanExecuteNext);
    PreviousCommand = new Command(ExecutePrevious, CanExecutePrevious);
    ToggleWatchedCommand = new Command(async () => await ExecuteToggleWatchedAsync(), () =>
        CurrentVideo != null && IsNotLoading);

    // התחלת טעינת הנתונים הא-סינכרונית
    load = LoadVideosAsync();
}

private async Task LoadVideosAsync()
{
    try
    {
        IsLoading = true;
        RefreshCommands();
    }
}

```

```

var loaded = await _videoService.GetAllVideosAsync();
_videos.Clear();
_videos.AddRange(loaded);

_currentIndex = 0;
UpdateCurrentVideo();
}
catch (Exception ex)
{
    CurrentPositionText = "שגיאה בטעינת הנתונים";
    // כאן ניתן לתעד את השגיאה ex.Message
}
finally
{
    IsLoading = false;
    RefreshCommands();
}
}

```

ובפעולת שינוי הסטטוס נעדכן גם את השירות:

CSHARP

```

private async Task ExecuteToggleWatchedAsync()
{
    if (CurrentVideo == null) return;

    IsLoading = true;
    RefreshCommands();

    CurrentVideo.IsWatched = !CurrentVideo.IsWatched;
    await _videoService.UpdateVideoAsync(CurrentVideo);

    OnPropertyChanged(nameof(CurrentVideo));

    IsLoading = false;
    RefreshCommands();
}

```

ובקובץ `MainPage.xaml.cs` נעדכן את שורת האתחול היחידה כדי להזריק את שירות המוק:

CSHARP

```

public MainPage()
{
    InitializeComponent();
    // של המסך BindingContext והגדרתו כ ViewModel-העברת מופע השירות לבנאי ה
    BindingContext = new MainViewModel(new MockVideoService());
}

```

צעד 4: מנגנון טעינת הנתונים וחיווי למשתמש – שימוש ב-ActivityIndicator וב-IsLoading

לפני שנעבור לחלק השני של המסך (טופס הוספת סרטון חדש), עלינו להשלים רכיב קריטי ביותר בארכיטקטורת האפליקציה: **חיווי על טעינת נתונים (Data Loading)**.

כאשר אפליקציית מובייל פונה לשירות נתונים (Service) – בין אם מדובר בשירות מקומי עם השהיה, במסד נתונים SQLite או בקריאת API לענן – הפעולה אורכת זמן ומתבצעת בצורה א-סינכרונית ברקע. אם המשתמש לוחץ על כפתור ודבר אינו משתנה במסך במשך שנייה או שתיים, נוצרת תחושת חוסר ודאות:

- המשתמש עלול לחשוב שהאפליקציה קפאה או קרסה.
- המשתמש עלול ללחוץ שוב ושוב על כפתורים (Spam clicks) ולגרום לשגיאות ולריבוי קריאות מיותרות.

כדי לפתור בעיה זו, אנו מחברים בין שני עולמות:

1. **בעולם ה-UI (ב-XAML):** הפקד הוויזואלי `ActivityIndicator` שמציג גלגל טעינה מסתובב.
2. **בעולם הלוגיקה (ב-ViewModel):** דגלי המצב `IsLoading` ו-`IsNotLoading` המנהלים את מחזור חיי הפעולה ומגנים על הפקודות מפני ביצוע כפול.

א. מהו הפקד `ActivityIndicator` ומה תפקידו?

ה-`ActivityIndicator` הוא פקד גרפי מובנה ב-.NET MAUI המיועד להציג למשתמש אנימציה של פעילות רקע מתמשכת (בצורת גלגל מסתובב – Spinner). שלא כמו מד התקדמות (`ProgressBar`) המציג אחוזים מדויקים (0% עד 100%), ה-`ActivityIndicator` מיועד לפעולות באורך לא-ידוע מראש (Indeterminate Progress) – בדיוק כמו טעינת רשימת סרטונים.

ב. תכונות המפתח של הפקד והשימוש ב-`IsLoading`

הפקד `ActivityIndicator` כולל 3 תכונות מרכזיות שבאמצעותן אנו שולטים בו מה-ViewModel:

תכונה בפקד (Property)	טיפוס	תפקיד בממשק	כיצד אנו קושרים אותה ב-XAML?
<code>IsRunning</code>	<code>bool</code>	קובע האם גלגל האנימציה מסתובב באופן פעיל	<code>{Binding IsLoading}</code>
<code>IsVisible</code>	<code>bool</code>	קובע האם הפקד מוצג על המסך או מוסתר לחלוטין	<code>{Binding IsLoading}</code>
<code>Color</code>	<code>Color</code>	קובע את צבע גלגל הטעינה (מותאם לשפת העיצוב)	<code>Color="#2563EB"</code> צבע מותאם, למשל



נקודה קריטית: מדוע חובה לקשור גם את `IsRunning` וגם את `IsVisible` ?

שימו לב לטעות נפוצה ביותר אצל מפתחים מתחילים: אם נקשור רק את `IsRunning="{Binding IsLoading}"` ונשכח לקשור את `IsVisible`:

- כאשר `IsLoading = true`: הגלגל יסתובב כרגיל.
- אך כאשר `IsLoading = false`: הגלגל יפסיק להסתובב אך יישאר תקוע על המסך כעיגול קפוא, או יתפוס מקום ריק ומכוער בתוך הפריסה! **הקישור הכפול מבטיח**: כאשר הטעינה מסתיימת, הגלגל גם מפסיק להסתובב וגם נעלם לחלוטין מהמסך (`IsVisible = false`) ומפנה את מקומו לתוכן הרגיל.

ג. מאפייני המצב ב-ViewModel: `IsLoading` ו-`IsNotLoading`

במחלקת הבסיס `BaseViewModel` הגדרנו שני מאפיינים שמשלימים זה את זה:

1. `IsLoading` (מאפיין בוליאני ראשי):

- דגל המציין האם מתבצעת כעת פעולת רקע.
- כאשר ערכו משתנה, ה-setter קורא פעמיים ל-`OnPropertyChanged`:
 - פעם אחת לעצמו: `OnPropertyChanged();` (מעדכן את ה-`ActivityIndicator`).
 - פעם שנייה עבור המאפיין הנגזר: `OnPropertyChanged(nameof(IsNotLoading));` (מעדכן את כל מי שתלוי בהיות המערכת פנויה).

2. `IsNotLoading` (מאפיין עזר נגזר):

- מוגדר כמאפיין קריאה בלבד: `public bool IsNotLoading => !IsLoading;`
- **היתרון הפדגוגי והמעשי**: במקום לסבך תלמידים עם ממירי ערכים הפוכים ב-XAML (`InvertedBoolConverter`), המאפיין הזה מאפשר לבדוק בצורה קריאה וישירה בתנאי הפקודות (`CanExecute`) האם המערכת אינה בטעינה.

ד. פירוק התוספות לבלוקים מודולריים

🧱 בלוק 4.1: בלוק טיפול ב-ViewModel (ניהול מחזור החיים והגנה על פקודות)

הטיפול ב-ViewModel מורכב משלושה מרכיבים:

1. הגדרת המאפיינים ב-`BaseViewModel`:

CSHARP

```
// מתוך BaseViewModel.cs:
private bool _isLoading;

public bool IsLoading
{
    get => _isLoading;
    set
    {

```

```

        if (_isLoading != value)
        {
            _isLoading = value;
            OnPropertyChanged();
            OnPropertyChanged(nameof(IsNotLoading));
        }
    }

    public bool IsNotLoading => !IsLoading;

```

2. תבנית מחזור החיים הא-סינכרוני (try-catch-finally): בכל פעולה שפונה לשירות (כמו `LoadVideosAsync` או `ExecuteToggleWatchedAsync`), מקפידים על התבנית:

- מדליקים `IsLoading = true;` ומרעננים פקודות ב- `RefreshCommands();`
- מבצעים את הפנייה לשירות בתוך בלוק `try`.
- מטפלים בחריגות בתוך בלוק `catch`.
- חובה: מכבים `IsLoading = false;` ומרעננים פקודות בתוך בלוק `finally`!

CSHARP

```

// מתוך MainViewModel.cs:
private async Task LoadVideosAsync()
{
    try
    {
        // 1. הרמת דגל טעינה ונטרול כפתורים
        IsLoading = true;
        RefreshCommands();

        // 2. ביצוע הפנייה לשירות הא-סינכרוני
        var loaded = await _videoService.GetAllVideosAsync();
        _videos.Clear();
        _videos.AddRange(loaded);

        _currentIndex = 0;
        UpdateCurrentVideo();
    }
    catch (Exception ex)
    {
        // 3. טיפול בשגיאות ותצוגת הודעה למשתמש
        CurrentPositionText = "שגיאה בטעינת הנתונים";
    }
    finally
    {
        // 4. כיבוי מובטח של דגל הטעינה ורענון מצב הכפתורים
        IsLoading = false;
        RefreshCommands();
    }
}

```



⚠ מדוע בלוק ה- `finally` הוא קריטי ומציל חיים?

קוד בתוך `finally` מתבצע תמיד – גם אם התרחשה תקלת רשת, שגיאת שרת או שנזרקה חריגה (`Exception`). אם היינו רושמים `IsLoading = false;` רק בסוף ה- `try`, במקרה של שגיאה שורת הכיבוי לעולם לא הייתה מתבצעת! התוצאה: ה- `ActivityIndicator` היה ממשיך להסתובב לעולמי עולמים, והמשתמש היה נשאר תקוע עם אפליקציה נעולה.

3. הגנה על פקודות באמצעות `IsNotLoading` ב- `CanExecute`: כדי למנוע מהמשתמש ללחוץ על כפתורי ניווט בזמן שהנתונים נטענים, אנו משלבים את `IsNotLoading` בתנאי הפקודות:

CSHARP

```
private bool CanExecuteNext()
{
    return IsNotLoading && _videos.Count > 0 && _currentIndex < _videos.Count - 1;
}

private bool CanExecutePrevious()
{
    return IsNotLoading && _videos.Count > 0 && _currentIndex > 0;
}
```

בזכות שילוב זה, ברגע שמופעלת מתודת `RefreshCommands()`, הכפתורים הופכים אוטומטית למנוטרלים (אפורים ולא לחיצים) כל עוד `IsLoading = true`!

🧱 בלוק 4.2: בלוק טיפול ב-XAML (מיקום והגדרת הפקד במסך)

כעת נחבר את הפקד לממשק המשתמש בקובץ `MainPage.xaml`:

1. **מיקום הפקד בעץ הרכיבים:** נמקם את ה- `ActivityIndicator` ישירות תחת כותרת הדף (`Label Title`) ומעל כרטיסיות התוכן בתוך ה- `VerticalStackLayout`. כך המשתמש רואה מיד במרכז המסך העליון שהאפליקציה עסוקה בטעינה.

2. **הגדרת הפקד ב-XAML (עם שבירת שורות תקנית לכל תכונה):**

XML

```
<!-- כותרת ראשית של המסך -->
<Label
    FontAttributes="Bold"
    FontSize="22"
    HorizontalOptions="Center"
    Text="{Binding Title}" />

<!-- ViewModel ב-IsLoading מחוץ טעינה א-סינכרוני המקושר לדגל -->
<ActivityIndicator
    HorizontalOptions="Center"
    IsRunning="{Binding IsLoading}" />
```

```
IsVisible="{Binding IsLoading}"
Color="#2563EB" />
```

3. כיצד ה-Binding עובד בפועל במסך?

- כאשר `IsLoading` הופך ל-`true` → ה-`ActivityIndicator` הופך גלוי (`IsVisible=True`), גלגל האנימציה מסתובב (`IsRunning=True`), וכפתורי הדפדוף מנוטרלים הודות ל-`IsNotLoading = false`.
- כאשר `IsLoading` חוזר ל-`false` → ה-`ActivityIndicator` נעלם (`IsVisible=False`), האנימציה נעצרת (`IsRunning=False`), והכפתורים חוזרים למצב פעיל.

טבלת סיכום: מנגנון ה-ActivityIndicator וה-ViewModel

רכיב במערכת	מיקום בקוד	תפקיד במנגנון הטעינה	תוצאה ויזואלית במסך
<code>IsLoading = true</code>	ViewModel (תחילת פעולה)	הרמת דגל טעינה	ה-ActivityIndicator הופך גלוי ומתחיל להסתובב
<code>RefreshCommands()</code>	ViewModel	רענון תנאי ה-CanExecute	כפתורי הניווט והפעולה הופכים לאפורים ומנוטרלים
<code>await Service ...</code>	ViewModel (בתוך <code>try</code>)	פנייה א-סינכרונית לשירות הנתונים	המשתמש רואה שהמערכת עובדת ולא נתקעה
<code>IsLoading = false</code>	ViewModel (בתוך <code>finally</code>)	הורדת דגל טעינה מובטחת	ה-ActivityIndicator נעלם לחלוטין מהמסך
<code>RefreshCommands()</code>	ViewModel (בתוך <code>finally</code>)	רענון תנאי ה-CanExecute מחדש	הכפתורים הרלוונטיים חוזרים להיות פעילים ולחיצים

שאלות הבנה ובדיקה עצמית – מנגנון טעינת נתונים

- מדוע חובה להגדיר ב-`ActivityIndicator` גם את `IsRunning` וגם את `IsVisible` ולא להסתפק באחד מהם?
- מה היה קורה באפליקציה אם היינו שוכחים להשתמש בבלוק `finally` והשירות היה זורק שגיאת רשת?
- כיצד המאפיין `IsNotLoading` מגן על האפליקציה מפני לחיצה כפולה מהירה (Double-Tap) של משתמש חסר סבלנות?

שלב ג': הרחבת האפליקציה – טופס הוספת סרטון חדש

כעת אנו מוכנים להוסיף את **בולק 2: כרטיסיית טופס הזנת סרטון חדש**, שוב בפירוק מודולרי מלא לבלוקים ובטבלאות הסבר.

צעד 1: הרחבת ה-ViewModel בשיטת הבלוקים

טבלת ניתוח שדות ופקודות הטופס (בלוק 2):

שדה / פקודה	טיפוס	פקד מקושר ב-XAML	כיוון קישור (Binding Mode)	תפקיד עסקית
InputTitle	string	Entry (כותרת)	TwoWay	קליטת כותרת הסרטון מהמשתמש בזמן אמת
InputChannelName	string	Entry (ערוץ)	TwoWay	קליטת שם הערוץ
InputDurationText	string	Entry (משך זמן)	TwoWay	קליטת משך הזמן כמחרוזת (מקלדת מספרית)
InputUrl	string	Entry (קישור)	TwoWay	קליטת כתובת ה-URL של הסרטון
StatusMessage	string	Label (הודעת סטטוס)	OneWay	הצגת משוב למשתמש (הצלחה או שגיאת קלט)
SaveCommand	ICommand	Button (שמור סרטון)	OneWay	ולידציה, הוספה לשירות, רענון ומעבר לסרטון החדש
ClearFormCommand	ICommand	Button (נקה טופס)	OneWay	איפוס מיידי של כל שדות הטופס

בלוק 1: שדות קלט בדידים (Two-Way Properties)

CSHARP

```
// שדות הטופס להזנת סרטון חדש (Two-Way Binding)
private string _inputTitle = string.Empty;
public string InputTitle
{
    get => _inputTitle;
    set
    {
        if (_inputTitle != value)
        {
            _inputTitle = value;
            OnPropertyChanged();
        }
    }
}

private string _inputChannelName = string.Empty;
public string InputChannelName
{
    get => _inputChannelName;
    set
    {
        if (_inputChannelName != value)
        {
            _inputChannelName = value;
            OnPropertyChanged();
        }
    }
}

private string _inputDurationText = string.Empty;
public string InputDurationText
{
    get => _inputDurationText;
```

```

    set
    {
        if (_inputDurationText ≠ value)
        {
            _inputDurationText = value;
            OnPropertyChanged();
        }
    }
}

private string _inputUrl = string.Empty;
public string InputUrl
{
    get ⇒ _inputUrl;
    set
    {
        if (_inputUrl ≠ value)
        {
            _inputUrl = value;
            OnPropertyChanged();
        }
    }
}

private string _statusMessage = string.Empty;
public string StatusMessage
{
    get ⇒ _statusMessage;
    set
    {
        if (_statusMessage ≠ value)
        {
            _statusMessage = value;
            OnPropertyChanged();
        }
    }
}

```

❏ בלוק 2: הגדרת הפקודות והבנאי

CSHARP

```

public ICommand SaveCommand { get; }
public ICommand ClearFormCommand { get; }

// בתוך הבנאי :
SaveCommand = new Command(async () ⇒ await ExecuteSaveAsync(), () ⇒ IsNotLoading);
ClearFormCommand = new Command(ExecuteClearForm, () ⇒ IsNotLoading);

```

CSHARP

```
private void RefreshCommands()
{
    ((Command)NextCommand).ChangeCanExecute();
    ((Command)PreviousCommand).ChangeCanExecute();
    ((Command)SaveCommand).ChangeCanExecute();
    ((Command)ToggleWatchedCommand).ChangeCanExecute();
    ((Command)ClearFormCommand).ChangeCanExecute();
}
```

בלוק 4: מתודות הביצוע (אימות קלט, שמירה ואיפוס)

CSHARP

```
private async Task ExecuteSaveAsync()
{
    // 1. בדיקת תקינות (Validation)
    if (string.IsNullOrEmpty(InputTitle))
    {
        StatusMessage = "שגיאה: חובה להזין כותרת לסרטון";
        return;
    }

    IsLoading = true;
    RefreshCommands();

    int duration = 0;
    int.TryParse(InputDurationText, out duration);

    var newVideo = new VideoItem
    {
        Title = InputTitle.Trim(),
        ChannelName = string.IsNullOrEmpty(InputChannelName) ? "ערוץ כללי" :
        InputChannelName.Trim(),
        Url = InputUrl.Trim(),
        DurationMinutes = duration,
        IsWatched = false
    };

    // 2. שמירה בשירות
    await _videoService.AddVideoAsync(newVideo);
    _videos.Add(newVideo);

    // 3. מעבר מידי לסרטון החדש שנוסף
    _currentIndex = _videos.Count - 1;
    UpdateCurrentVideo();

    // 4. איפוס הטופס ועדכון הודעת סטטוס
    ExecuteClearForm();
    StatusMessage = $"נוסף בהצלחה לרשימה '{newVideo.Title}' הסרטון";
}
```

```
IsLoading = false;
RefreshCommands();
}

private void ExecuteClearForm()
{
    InputTitle = string.Empty;
    InputChannelName = string.Empty;
    InputUrl = string.Empty;
    InputDurationText = string.Empty;
}
```

צעד 2: הרכבת בלוק 2 ב-XAML בתת-בלוקים מודולריים

נפתח את `MainPage.xaml` ונרכיב את כרטיסיית הטופס מתחת לבלוק 1 בתת-בלוקים:

+ הוספת סרטון חדש (טופס קלט)

כותרת הסרטון:

פקודות ICommand ו-CanExecute

שם הערוץ:

Microsoft Developers

משך בדקות:

18

כתובת אינטרנט (URL):

<https://youtube.com/watch?v=demo2>

נקה טופס 🖌️

שמור סרטון 💾

הסרטון 'ICommand' נוסף בהצלחה לרשימה!

♦ תת-בלוק 2.1: מעטפת הכרטיסייה והכותרת

XML

```
<!-- כרטיסייה 2: טופס חונה של פריט בודד -->
<Border Padding="14"
  StrokeShape="RoundRectangle 8">
  <VerticalStackLayout Spacing="8">

    <Label Text="(טופס קלט) הוספת סרטון חדש"
      FontSize="16"
      FontAttributes="Bold" />
```

♦ תת-בלוק 2.2: שדות הקלט בקישור דו-כיווני (`Mode=TwoWay`)

שימו לב לשימוש ב- `Mode=TwoWay` ובמקלדת מספרית עבור שדה הדקות:

XML

```
<Label Text="כותרת הסרטון:" />
<Entry Text="{Binding InputTitle, Mode=TwoWay}"
  Placeholder="הגן כותרת" />

<Label Text="שם הערוץ:" />
<Entry Text="{Binding InputChannelName, Mode=TwoWay}"
  Placeholder="הגן ערוץ" />

<Label Text="משך בדקות:" />
<Entry Text="{Binding InputDurationText, Mode=TwoWay}"
  Placeholder="למשל 15"
  Keyboard="Numeric" />

<Label Text="(URL): כתובת אינטרנט:" />
<Entry Text="{Binding InputUrl, Mode=TwoWay}"
  Placeholder="https:// ... " />
```

♦ תת-בלוק 2.3: כפתורי פעולה והודעת סטטוס

גריד עבור כפתור שמירה וכפתור איפוס, ותווית המציגה את הודעת הסטטוס:

XML

```
<!-- כפתורי שמירה ואיפוס -->
<Grid Margin="0,8,0,0"
  ColumnDefinitions="*,*"
  ColumnSpacing="10">
  <Button Grid.Column="0"
    Text="שמור סרטון"
    Command="{Binding SaveCommand}" />

  <Button Grid.Column="1"
    Text="נקה טופס"
    Command="{Binding ClearFormCommand}" />
</Grid>

<!-- הודעת סטטוס משוב למשתמש -->
<Label Text="{Binding StatusMessage}"
  TextColor="DarkSlateBlue"
  HorizontalOptions="Center" />

</VerticalStackLayout>
</Border>
```



פינת העמקה: מהו Two-Way Data Binding ומדוע הוא הכרחי בטפסים? 📝

בעוד שבתוויות רגילות (Label) הנתונים זורמים רק מכיוון הקוד אל המסך (One-Way), בשדות קלט (Entry) המשתמש מקליד תוכן בעצמו. הגדרת Mode=TwoWay גורמת לכך שכל שינוי בטקסט במסך יעדכן בו-ברגע את המאפיין המתאים ב-ViewModel. כשהמשתמש לוחץ על "שמור סרטון", הנתונים כבר שמורים בזיכרון של ה-ViewModel ומוכנים לשמירה!

שאלות הבנה לשלב ג': 💡

- מה היה קורה בעת לחיצה על "שמור סרטון" אילו היינו משמיטים את Mode=TwoWay משדות ה-Entry?
- מדוע חשוב לבצע בדיקת תקינות (Validation) של שדות החובה ב-ViewModel לפני שפונים ל-Service?

צעד 3: בדיקות אינטגרציה מקצה לקצה

נריך כעת את האפליקציה המלאה:

- נקליד בטופס סרטון חדש: "פיתוח אפליקציות עם SQLite", ערוץ "25", Master C# דקות.
- נלחץ על "שמור סרטון".
- נראה שהשדות מתנקים מיד, מופיעה הודעת סטטוס משמחת, והתצוגה בבלוק העליון קופצת מיד לסרטון החדש ("סרטון 4 מתוך 4")!
- נדפדף אחורה עם "← הקודם" – כל הסרטונים הקודמים קיימים ושמורים.

נספח א': מדריך לימודי שלם על בדיקות יחידה (Unit Tests) 🧪

א.1 מהי בדיקת יחידה (Unit Test) ומדוע היא מפתח לאיכות?

בעולם התוכנה המקצועי, **בדיקת יחידה** היא קוד בדיקה קצר שבודק רכיב בודד ומבודד במערכת (כגון מחלקת ViewModel) ומוודא שהוא מתנהג בדיוק לפי הדרישות.

למה זה חשוב במיוחד ב-MVVM?

היופי הגדול בארכיטקטורת MVVM הוא **הפרדת הלוגיקה מהממשק הגרפי**. אם היינו כותבים את כל הקוד בתוך Button_Clicked במסך, הדרך היחידה לבדוק האם הדפדוף עובד הייתה להפעיל אמולטור כבד, לחכות 2 דקות, ללחוץ פיזית על הכפתורים, ולבדוק ידנית. בזכות ה-ViewModel, **ניתן לבדוק את כל הלוגיקה של הדפדוף, האינדקסים ותנאי ה-CanExecute בשבירר שנייה בקוד בדיקה אוטומטי – בלי להרים מסך כלל!**

א.2 תבנית AAA (Arrange - Act - Assert)

כל בדיקת יחידה מקצועית בנויה לפי 3 שלבים מוגדרים:

- Arrange (הכנה):** יצירת האובייקטים, אתחול המשתנים והכנת המצב הראשוני לבדיקה.
- Act (פעולה):** הפעלת המתודה או הפקודה שאותה אנו מעוניינים לבדוק (למשל `NextCommand.Execute()`).

3. **Assert (אימות):** בדיקה שהתוצאה שהתקבלה בפועל זהה לחלוטין לתוצאה הצפויה (אם שווה – הבדיקה עברה בהצלחה; אם לא – נכשלה ומציגה הסבר).

3.א הדגמה מעשית: בדיקת יחידה ל- `MainViewModel` משלב א'

נוסיף פרויקט בדיקות מסוג **xUnit** ונכתוב בדיקות יחידה מלאות לבדיקת הדפדוף ו- `CanExecute`:

CSHARP

```
using Xunit;
using PlaylistSample.ViewModels;

namespace PlaylistSample.Tests;

public class MainViewModelTests
{
    [Fact]
    public void Constructor_InitializesWithFirstVideo_AndPreviousCommandIsDisabled()
    {
        // 1. Arrange & Act: יצירת ViewModel
        var vm = new MainViewModel();

        // 3. Assert: אימות שהסרטון הראשון מוצג ושהכפתור הקודם מנוטרל
        Assert.NotNull(vm.CurrentVideo);
        Assert.Equal("סרטון 1 מתוך 3", vm.CurrentPositionText);
        Assert.False(vm.PreviousCommand.CanExecute(null), "!בסרטון הראשון כפתור הקודם חייב להיות מנוטרל");
        Assert.True(vm.NextCommand.CanExecute(null), "!בסרטון הראשון כפתור הבא חייב להיות פעיל");
    }

    [Fact]
    public void ExecuteNext_AdvancesToNextVideo_AndUpdatesCurrentVideo()
    {
        // 1. Arrange: ViewModel הכנת
        var vm = new MainViewModel();

        // 2. Act: הפעלת פקודת דפדוף קדימה
        vm.NextCommand.Execute(null);

        // 3. Assert: אימות שהאינדקס והמאפיינים התעדכנו לסרטון השני
        Assert.NotNull(vm.CurrentVideo);
        Assert.Equal("סרטון 2 מתוך 3", vm.CurrentPositionText);
        Assert.Equal("MVVM הבנת ארכיטקטורת", vm.CurrentVideo.Title);
        Assert.True(vm.PreviousCommand.CanExecute(null), "!בסרטון השני כפתור הקודם חייב להיות פעיל");
    }

    [Fact]
    public void ExecuteNext_AtEndOfList_DisablesNextCommand()
    {
        // 1. Arrange: ViewModel הכנת
        var vm = new MainViewModel();

        // 2. Act: דפדוף פעמיים עד לסוף הרשימה
        vm.NextCommand.Execute(null);
        vm.NextCommand.Execute(null);
    }
}
```

```
// 3. Assert: אימות שהגענו לסוף ושהכפתור הבא מנוטרל
Assert.NotNull(vm.CurrentVideo);
Assert.Equal("סרטון 3 מתוך 3", vm.CurrentPositionText);
Assert.False(vm.NextCommand.CanExecute(null), "!בסרטון האחרון כפתור הבא חייב להיות מנוטרל");
Assert.True(vm.PreviousCommand.CanExecute(null), "!בסרטון האחרון כפתור הקודם חייב להיות פעיל");
}
}
```

שימו לב לעוצמה של שיטה זו: בתוך 0.05 שניות, הבדיקות מוודאות שלעולם לא תהיה חריגה מגבולות הרשימה ושמצב הכפתורים מוגן ב-100%!

שאלות הבנה לנספח בדיקות היחידה:

1. מדוע קל לבדוק מחלקת ViewModel בבדיקת יחידה אוטומטית לעומת קוד שנכתב ב-Code-Behind של דף?
2. מה תפקידו של שלב ה-Assert בתבנית AAA?

נספח ב': מקורות השראה, תיעוד וביבליוגרפיה

מדריך זה פותח בהשראת מתודולוגיית ההוראה המודולרית **The Block Strategy** של **Leomaris Reyes** (Microsoft MVP), בלוגרית Telerik ויוצרת AskXammy.com, המנגישה עיצובי ממשק מובייל מורכבים ב-.NET MAUI בצורה הדרגתית, חזותית ונעימה.

תיעוד רשמי להעמקה נוספת:

- [Microsoft Learn: Data Binding & MVVM in .NET MAUI](#)
- [Microsoft Learn: String formatting in .NET MAUI Data Binding](#)
- [Microsoft Learn: The Command Interface & CanExecute](#)
- [Microsoft Learn: Compiled Bindings with x:DataType](#)
- [Microsoft Learn: Unit Testing C# with xUnit and .NET Core](#)